Litebee IDE · LiteBee Wing · набор FM-10

Программируем шоу дронов

Учебник по симулятору и первым полётам

Для школьников и учителей, которые впервые пишут шоу для нескольких дронов на Python и смотрят результат в симуляторе.

Все примеры и решения заданий лежат в папке `docs/uchebnik/solutions` и проверены загрузчиком и проверками симулятора.

Оглавление

Как пользоваться учебником 5

1. Знакомство с окном 6
- 1.1. Что мы делаем . . . 6
- 1.2. Начало работы . . . 6
- 1.3. Одно окно . . . 7
- 1.4. Порядок работы . . . 9
- 1.5. 3D . . . 9
- 1.6. Красная кнопка . . . 10

2. Первый взлёт 12
- 2.1. Самое короткое шоу . . . 12
- 2.2. Где писать программу . . . 12
- 2.3. Шаги и секунды . . . 13
- 2.4. Лимит высоты . . . 13

3. Свет 15
- 3.1. Команда rgb . . . 15
- 3.2. rgb занимает ноль секунд . . . 15
- 3.3. Свой цвет . . . 16

4. Высота и ожидание 17
- 4.1. Команда set_z . . . 17
- 4.2. Ожидание . . . 17
- 4.3. Что нельзя . . . 17

5. Коврик и клетки 19
- 5.1. Коврик 12 на 12 . . . 19
- 5.2. Номер клетки . . . 19
- 5.3. Четверть . . . 19
- 5.4. Метки на коврике . . . 19

6. Перелёт в клетку 22
- 6.1. Команда code . . . 22
- 6.2. Скорость выбираете вы . . . 22

7. Маршрут из нескольких клеток 24
- 7.1. Квадрат . . . 24
- 7.2. Как выбирать время . . . 24

8. Посадка в клетку 26
- 8.1. land и land_on . . . 26
- 8.2. Что делает land_on . . . 26

9. Смотрим шоу: партитура 28
- 9.1. Время одно на всё окно . . . 28

9.2. Партитура . . . 28
9.3. Как двигаться по времени . . . 30
9.4. Выбранный дрон и его код . . . 30

10. Два дрона 32
10.1. Два блока . . . 32
10.2. Правила безопасной дистанции . . . 33

11. Проверки 35
11.1. Что проверяет симулятор . . . 35
11.2. Читаем сообщение . . . 36
11.3. Три проблемы сразу . . . 36
11.4. Лимит высоты и область . . . 37

12. Очередь и ожидание 39
12.1. У дронов нет общих часов . . . 39
12.2. Эстафета . . . 39

13. Повторы: циклы 42
13.1. Цикл в программе одного дрона . . . 42
13.2. Цикл по списку . . . 42
13.3. Цикл по дронам . . . 43

14. Другая четверть 44
14.1. Сдвиг номеров . . . 44
14.2. Перенос в меню «Шоу…» . . . 45

15. Круги и подъём на ходу 47
15.1. Круг: circle . . . 47
15.2. Подъём на ходу: code_z . . . 48

16. Итоговые проекты 49
16.1. Проект 1. За лидером (2 дрона) . . . 49
16.2. Проект 2. Лесенка (3 дрона) . . . 49
16.3. Проект 3. Четыре угла (4 дрона) . . . 49
16.4. Проект 4. Пятёрка (5 дронов) . . . 50
16.5. Проект 5. Ряды меняются (6 дронов) . . . 50
16.6. Проект 6. Семёрка (7 дронов) . . . 50
16.7. Проект 7. Хоровод (8 дронов) . . . 50

17. От симулятора к дрону 51
17.1. Цепочка перед полётом . . . 51
17.2. Связь и дроны . . . 51
17.3. Батарея . . . 52
17.4. Позиция по коврику . . . 52
17.5. Что полетит не так, как на экране . . . 53
17.6. Аварийная посадка . . . 53

A. Таблица команд 54

B. Частые ошибки 56

C. Глоссарий **58**

D. Что ещё не проверено **60**

E. Ответы к упражнениям **61**

Урок 1. Знакомство с окном . . . 61
Урок 2. Первый взлёт . . . 61
Урок 3. Свет . . . 62
Урок 4. Высота и ожидание . . . 64
Урок 5. Коврик и клетки . . . 65
Урок 6. Перелёт в клетку . . . 65
Урок 7. Маршрут из нескольких клеток . . . 67
Урок 8. Посадка в клетку . . . 68
Урок 9. Смотрим шоу: партитура . . . 69
Урок 10. Два дрона . . . 70
Урок 11. Проверки . . . 72
Урок 12. Очередь и ожидание . . . 73
Урок 13. Повторы: циклы . . . 76
Урок 14. Другая четверть . . . 77
Урок 15. Круги и подъём на ходу . . . 78
Урок 16. Итоговые проекты . . . 79

Как пользоваться учебником

Эта книга учит описывать шоу дронов программой на Python и проверять его в симуляторе. Дроны для этого не нужны: всё происходит на экране. Когда шоу готово и проверено, его можно загрузить в настоящие дроны LiteBee Wing (об этом последний урок).

Как устроен каждый урок. Он начинается с самого простого. Сначала небольшой рабочий пример, потом объяснение, что в нём происходит, и в конце рамка **«Попробуйте сами»**: несколько упражнений, чтобы закрепить. Ответы ко всем упражнениям в конце книги. Сначала попробуйте сами, потом подсматривайте.

Как работать.

1. Возьмите пример из урока. Каждый листинг лежит в папке `docs/uchebnik/solutions` под тем именем, которое написано над ним.
2. Откройте файл в Litebee IDE (на стартовом экране «Загрузить файл с компьютера») и нажмите «Проверить», потом «Играть». Посмотрите, что получилось.
3. Поменяйте число или строку, нажмите «Проверить» ещё раз. Так учишься быстрее всего.
4. Выполните упражнения, сверьтесь с ответами.

Что проверено. Все примеры и ответы к упражнениям (файлы `l*.py` и `u*.py`) прошли проверки симулятора без предупреждений и ошибок. Файлы, которые нарочно ошибаются, лежат в `solutions/bad_examples`: на них показаны настоящие сообщения.

> **Значки в тексте**
>
> В такой рамке лежит то, что стоит запомнить.

> **Осторожно**
>
> Так отмечено то, что легко сделать неправильно.

> **Безопасность**
>
> Так отмечено то, что касается настоящих дронов.

Что ещё не проверено на живом дроне. Шаг `code_z` приложение упаковывает для загрузки так же, как LiteBee Go, но на настоящем дроне он пока не летал. Шаг `circle` летал, но время круга на дроне и в симуляторе разное. Шаг `circle_z` в дрон загрузить нельзя вообще. Эти места помечены в тексте.

Урок 1

Знакомство с окном

Что вы узнаете: как начать проект, из чего состоит окно Litebee IDE, как запустить готовое шоу и где находится красная кнопка.

1.1. Что мы делаем

Шоу дронов — это программа для нескольких дронов сразу. У каждого дрона своя короткая программа из шагов: откалибровался, взлетел, зажёг свет, перелетел в другую клетку, сел. В приложении дрон шоу называется **бортом**: борт 1, борт 2 и так далее. Программы мы пишем на Python. Слово **симулятор** значит само окно Litebee IDE: оно читает программы и показывает карту с летающими дронами, временную ленту и список проверок.

Программа не вычисляет ничего сложного. Она просто *записывает шаги*, а симулятор их проигрывает. Настоящий дрон нельзя перезапустить после ошибки, поэтому сначала шоу проверяют на экране.

1.2. Начало работы

При запуске приложение показывает стартовый экран с тремя карточками.

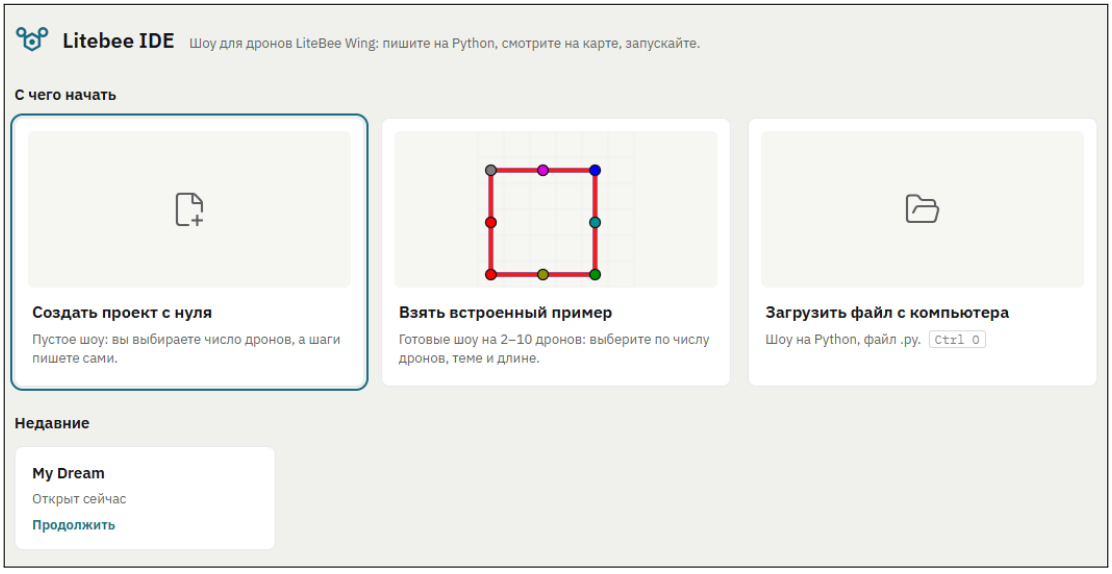


Рис. 1.1.. Стартовый экран. Три способа начать и недавние проекты. Внизу справа переключатели темы и языка (RU / EN).

- «Создать проект с нуля»: пустое шоу, вы выбираете число дронов, а шаги пишете сами.

- «Взять встроенный пример»: готовые шоу на 2–10 дронов. Открывается окно «Новый проект» с карточками примеров, у каждой миниатюра маршрута (рисунок 1.2). Слева фильтры: карта, число дронов, тема, длительность, особенности.
- «Загрузить файл с компьютера» (Ctrl + O): шоу на Python, файл `.py`. Так открываются файлы этой книги из папки `docs/uchebnik/solutions`.

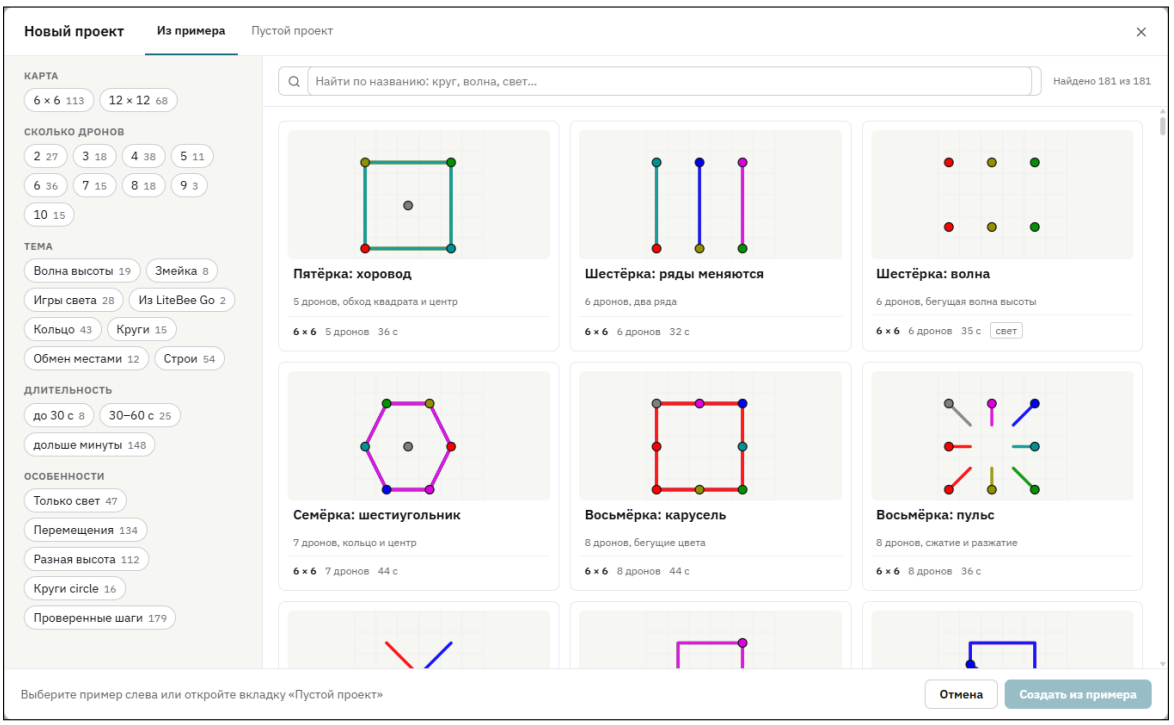


Рис. 1.2.. Окно «Новый проект», вкладка «Из примера»: слева фильтры, справа карточки примеров. Есть и вкладка «Пустой проект».

> **Что вы видите в программе**
>
> В приложении вы видите только программу выбранного дрона: строки, которые начинаются с `d = show.drone(N)`. Две строки в начале файла (`from litebee_show import Show` и `show = Show('имя')`) приложение показывает не всегда, но они есть в файле. В этой книге каждый листинг напечатан *целиком*, как файл лежит на диске.

1.3. Одно окно

Окно разделено на две части. Слева **сцена**: коврик с дронами. Справа **рабочая панель** с вкладками. Сверху шапка. У левого края окна прячется боковая панель: при наведении мыши она выезжает.

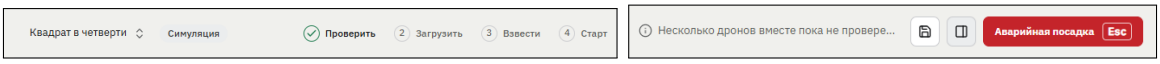


Рис. 1.3.. Шапка окна. Слева название проекта и его состояние («Симуляция»), потом четыре шага запуска. Справа важное предупреждение, кнопка «Сохранить», кнопка «Инспектор» и красная «Аварийная посадка».

Шапка

- **Название проекта** и состояние: «Симуляция», «Взведено» или «Летит». Щелчок по названию открывает меню проекта: начальный экран, новый проект, открыть, сохранить.
- **Четыре шага** «Проверить», «Загрузить», «Взвести», «Старт». Первый проверяет шоу на экране, остальные нужны только для настоящих дронов (урок 17).
- **Одна строка предупреждения**: самое важное, что приложение хочет вам сказать. Например «Несколько дронов вместе пока не проверены в полёте».
- Кнопки **Сохранить** и **Инспектор** (показать или спрятать правую панель).
- **Аварийная посадка** с надписью `Esc`: красная кнопка всегда на виду.

Сцена слева

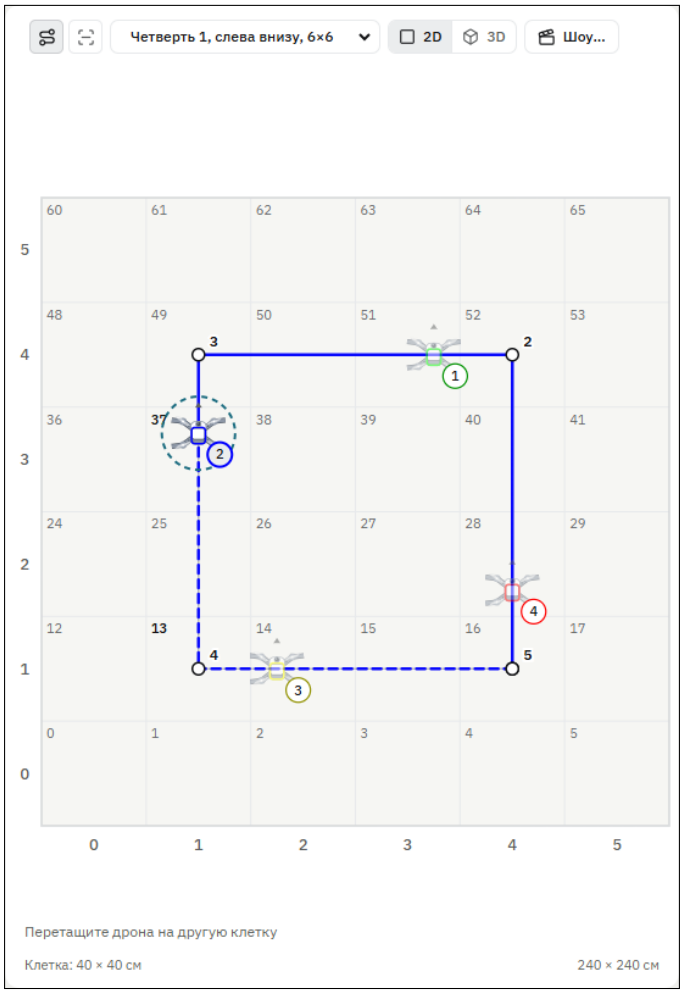


Рис. 1.4.. Сцена: карта сверху. Вверху две кнопки (показать маршруты, показать метки), выбор области, переключатель 2D / 3D и меню «Шоу…». Дроны в кружках с номерами, пунктиром и сплошной линией показан маршрут.

На сцене коврик с клетками, на нём дроны (в кружке номер борта) и их маршруты. Вверху: две кнопки (показать маршруты, показать метки коврика), выбор «Область» (весь коврик или одна четверть), переключатель «2D» / «3D» и меню «Шоу…». В меню «Шоу…» лежат перенос шоу в другую четверть (урок 14) и высота (лимит высоты, урок 2).

Дрона можно взять мышью и перетащить на другую клетку: приложение само поправит стартовую клетку в программе. Щелчок по дрону выбирает его и открывает его программу.

Рабочая панель справа

Над вкладками общее управление временем: кнопка «Играть», скорость (1×, 4×, 10×), время вида «20,0 / 30,0 с» и ползунок. Время одно на всё окно. Вкладки идут в один ряд:

- **Партитура**: временная лента по каждому дрону (урок 9).
- **Код**: программа выбранного дрона (урок 2).
- **Борты**: настоящие дроны на связи (урок 17).
- **Проверки**: список замечаний (урок 11).
- **Справочник**: все команды с примерами. Справа от команды отмечено, проверена ли она на живом дроне, а кнопка «Вставить в код» добавляет пример в программу.

1.4. Порядок работы

1. Создайте проект и расставьте дронов по клеткам: перетащите их мышью.
2. Напишите программу каждого дрона (вкладка «Код»).
3. Нажмите «Проверить», посмотрите вкладку «Проверки».
4. Нажмите «Играть» и посмотрите шоу на сцене и в партитуре.
5. Только потом (с настоящими дронами) «Загрузить», «Взвести», «Старт».

1.5. 3D

Кнопка «3D» над сценой включает объёмный вид. Дроны показаны как настоящие LiteBee Wing: светодиод горит цветом шоу, в воздухе вращаются пропеллеры, над дроном плашка с номером. На полу метки AprilTag. Камеру можно вращать мышью (колесо меняет масштаб, правая кнопка двигает).

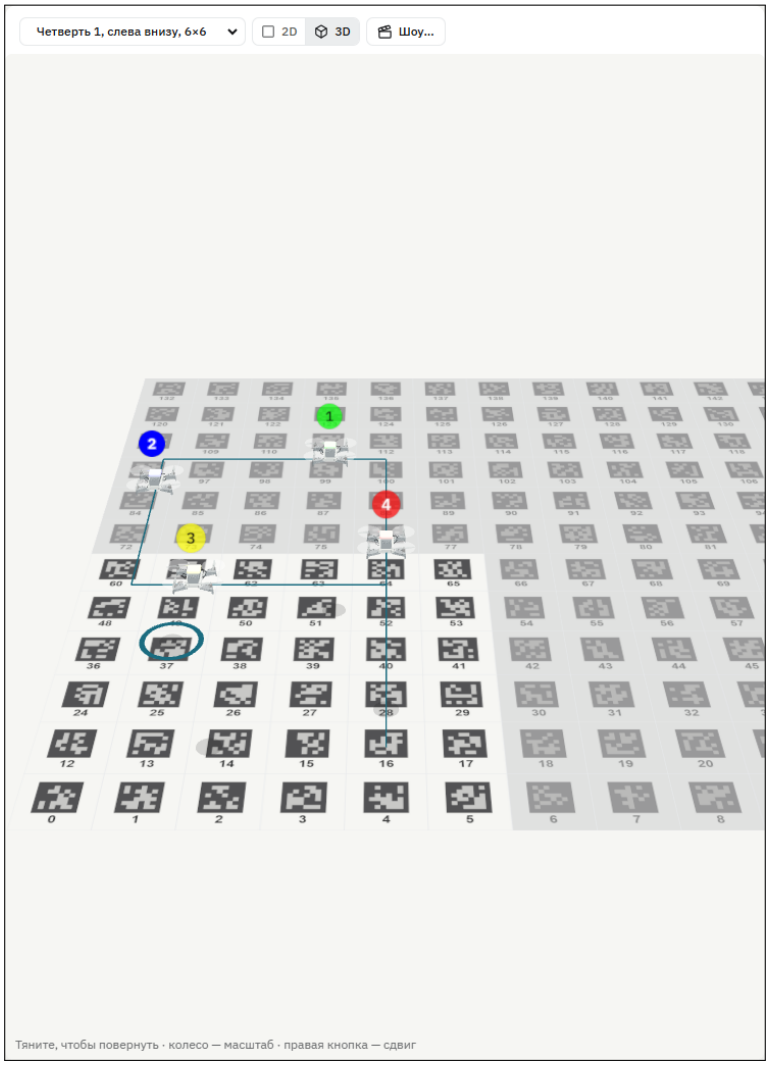


Рис. 1.5.. Сцена в режиме 3D. Видна высота полёта, метки на полу и плашки дронов.

1.6. Красная кнопка

Красная кнопка «Аварийная посадка» всегда на месте, в правом верхнем углу. Ей соответствует клавиша `Esc`. Она посылает команду посадки всем дронам на связи. Пока дронов нет, она просто останавливает симуляцию.

> **Безопасность**
>
> Красная кнопка и `Esc` работают в любой момент. Перед настоящим полётом убедитесь, что знаете, где она, и что рука может до неё дотянуться.

Попробуйте сами

- **1.1** Возьмите встроенный пример «Квадрат» и нажмите «Играть». Сколько секунд длится шоу?
- **1.2** Щёлкните по дрону 3 на карте. Какого цвета у него светодиод?

1.3 Щёлкните по дрону 4. На какой клетке он стоит?

1.4 Нажмите `Esc`. Что написано в сообщении?

Ответы к упражнениям этого урока на странице 61.

Урок 2

Первый взлёт

Что вы узнете: как выглядит самое короткое шоу, что такое шаг, откуда берутся секунды.

2.1. Самое короткое шоу

solutions/l02_first.py

```
# Калибровка, взлёт на 100 см и посадка.
from litebee_show import Show

show = Show('Первый взлёт')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.land()
```

Разберём построчно.

- `from litebee_show import Show` берёт из библиотеки слово `Show`. Эта строка всегда первая.
- `show = Show('Первый взлёт')` создаёт шоу и даёт ему имя. Переменная *обязана* называться `show`: приложение берёт шоу именно из неё.
- `d = show.drone(1)` создаёт дрона с номером 1. Дальше `d` — это «наш дрон»: команды вида `d.что-то(...)` добавляют ему шаги.
- `d.calibrate(5)`: калибровка, 5 секунд. Обычно это первый шаг.
- `d.takeoff(100, 5)`: взлёт на высоту 100 см за 5 секунд.
- `d.land()`: посадка.

Строки, которые начинаются с #, — комментарии. Python их пропускает, они для людей.

2.2. Где писать программу

Откройте проект и щёлкните по дрону на карте (или по его номеру в партитуре). Справа откроется вкладка «Код» с программой *этого* дрона. Над программой написано, на какой клетке стоит дрон, и есть кнопка «Выбрать клетку». Все команды, которые мы разбираем в книге, вы пишете именно здесь, по одной на строку. Когда меняете код, приложение само запускает его заново и обновляет карту и партитуру.

2.3. Шаги и секунды

Каждая команда добавляет дрону один **шаг**. Шаги выполняются строго по порядку: следующий начинается, когда закончился предыдущий. Число секунд в команде — это **длительность шага**, а не момент времени. `d.takeoff(100, 5)` значит «взлетай пять секунд», а не «будь наверху на пятой секунде».

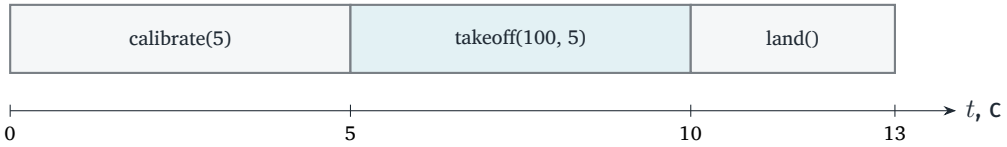


Рис. 2.1.. Лента времени шоу `l02_first.py`: $5 + 5 + 3 = 13$ секунд.

Время программы дрона — сумма времён шагов. Посадка в симуляторе всегда занимает 3 секунды, хотя в скобках у `land()` ничего не написано.

> **Запомните**
>
> У каждого дрона свои часы, они начинаются с нуля. Дрон не знает, что делают другие (урок 12).

2.4. Лимит высоты

Высота — расстояние от пола до дрона в сантиметрах. Выше лимита лететь нельзя: по умолчанию лимит 150 см. Его можно поменять в меню «Шоу...» над сценой, в разделе «Высота» (от 20 до 200 см). Посмотрите, что скажет проверка, если взлететь слишком высоко:

`solutions/bad_examples/bad_height.py`

```
from litebee_show import Show

show = Show('Слишком высоко')

d = show.drone(1)
d.calibrate(5)
d.takeoff(180, 5)
d.land()
```

Нажмите «Проверить»: в списке появится ошибка «Дрон 1: высота 180 см больше лимита 150 см, полёт будет отклонён».

> **Попробуйте сами**
>
> **2.1** Взлетите на 60 см за 4 секунды и сразу сядьте.
>
> **2.2** Добавьте зелёный свет (`d.rgb('green')`), чтобы дрон повисел 3 секунды перед посадкой. Подсказка: ожидание записывается как `d.set_z(100, 3)` (о нём урок 4); потом верните светодиод в `'black'`.
>
> **2.3** Сколько секунд длится шоу из файла `l02_first.py`, если взлёт записать как

`takeoff(100, 8)`?

2.4 Что нужно поменять в файле `bad_height.py`, чтобы проверка прошла?

Ответы к упражнениям этого урока на странице 61.

Урок 3

Свет

Что вы узнете: как менять цвет светодиода и почему цвет нужно «удерживать».

3.1. Команда rgb

На каждом дроне есть цветной светодиод. Цвет меняет команда `rgb`. Вот шоу, в котором дрон по очереди горит красным, зелёным и синим:

solutions/l03_light.py

```
# Каждый цвет держится две секунды.
from litebee_show import Show

show = Show('Три цвета')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('red')
d.set_z(100, 2)
d.rgb('green')
d.set_z(100, 2)
d.rgb('blue')
d.set_z(100, 2)
d.rgb('black')
d.land()
```

Цвет можно задать *именем*. Всего восемь имён, у каждого есть номер:

0	1	2	3	4	5	6	7
`black`	`red`	`green`	`yellow`	`blue`	`purple`	`cyan`	`white`

Цвет `black` выключает светодиод. Вместо имени можно написать номер: `d.rgb(4)` — это синий.

3.2. rgb занимает ноль секунд

Это главное, что нужно запомнить про свет. Команда `rgb` не длится: цвет меняется в этот момент и держится до следующего `rgb`. Чтобы цвет можно было увидеть, после него нужен шаг, у которого есть длительность. В примере это `d.set_z(100, 2)`: дрон остаётся на высоте 100 см две секунды и светится.

Осторожно

Если написать два `rgb` подряд, первый цвет никто не увидит: между ними прошло ноль секунд.

3.3. Свой цвет

Кроме имён, цвет можно задать тремя числами от 0 до 255: яркость красного, зелёного и синего. Для этого есть `d.rgb_value(255, 128, 0)` или `d.rgb((255, 128, 0))` (это то же самое). Эта смесь даёт оранжевый.

Запомните

На живых дронах проверены только восемь именованных цветов. Свой цвет в симуляторе работает, но на дроне может выглядеть иначе.

Если написать неизвестное имя, например `d.rgb('pink')`, будет ошибка: «Неизвестный цвет 'pink'. Доступны: black, red, green, yellow, blue, purple, cyan, white».

Попробуйте сами

3.1 Светофор: красный 3 секунды, жёлтый 1 секунда, зелёный 3 секунды. Дрон висит на высоте 60 см. Сколько секунд длится шоу?

3.2 Радуга: покажите по очереди цвета с номерами от 1 до 7, по секунде каждый. Используйте номера, а не имена.

3.3 Сделайте оранжевый свет через `rgb_value(255, 128, 0)` на 3 секунды.

3.4 Что увидит зритель в таком коде? Почему?

```
d.rgb('red')  d.rgb('green')  d.set_z(100, 2)
```

Ответы к упражнениям этого урока на странице 62.

Урок 4

Высота и ожидание

Что вы узнете: как менять высоту и как заставить дрон ждать.

4.1. Команда set_z

`d.set_z(z, seconds)` переводит дрона на высоту `z` сантиметров за `seconds` секунд. Дрон при этом остаётся над той же клеткой.

solutions/l04_height.py

```
# Подъём, пауза на новой высоте и спуск.
from litebee_show import Show

show = Show('Этажи')

d = show.drone(1)
d.calibrate(5)
d.takeoff(60, 5)
d.set_z(120, 3)
d.set_z(120, 2)
d.set_z(60, 3)
d.land()
```

Что происходит: взлёт на 60 см, подъём до 120 см (3 с), потом ещё 2 с на той же высоте, спуск до 60 см (3 с) и посадка.

4.2. Ожидание

Отдельной команды «ждать» нет. Если надо подождать, дрону говорят: «перейди на ту высоту, на которой ты уже висишь». Вторая строка `d.set_z(120, 2)` в примере выше и есть ожидание: высота не меняется, проходят две секунды.

> **Запомните**
>
> Ожидание = `set_z` на той же высоте. Это основной приём для шоу из нескольких дронов (урок 12).

4.3. Что нельзя

- Высота не больше лимита (по умолчанию 150 см, максимум 200 см).
- Одно ожидание не длиннее 60 секунд. Если нужно дольше, разбейте на несколько шагов. Эта проверка срабатывает при загрузке в дрон.

Попробуйте сами

4.1 Лесенка вверх: взлёт на 50 см, потом ступени 100 см и 150 см, по 2 секунды каждая.

4.2 «Дыхание»: между 60 см и 120 см два раза туда и обратно, по 2 секунды на каждый переход.

4.3 Дрон висит 10 секунд на 100 см: первые пять секунд светится синим, следующие пять красным.

4.4 «Лифт»: взлёт на 150 см, спуск до 30 см за 4 секунды, потом подъём до 100 см за 3 секунды.

Ответы к упражнениям этого урока на странице 64.

Урок 5

Коврик и клетки

Что вы узнете: как устроен коврик, как узнать номер клетки и что такое четверть. В этом уроке нет программ, зато он нужен для всех следующих.

5.1. Коврик 12 на 12

Дроны летают над ковриком с метками. Дрон смотрит вниз, видит метки под собой и по ним понимает, где он находится. Коврик поделён на квадратные клетки: **12 по горизонтали и 12 по вертикали**. Сторона клетки **40 см**, значит весь коврик 4,8 м на 4,8 м, всего 144 клетки с номерами от 0 до 143.

5.2. Номер клетки

У каждой клетки есть *столбец* (слева направо, от 0 до 11) и *строка* (снизу вверх, от 0 до 11). Номер считается так:

$$\text{номер} = \text{столбец} + 12 \times \text{строка}.$$

Клетка 97: столбец 1, строка 8, потому что $1 + 12 \times 8 = 97$. Обратно: разделите номер на 12, целая часть — строка, остаток — столбец. $97 = 12 \times 8 + 1$.

Клетка 0 внизу слева. Ось x идёт вправо, ось y вверх. Дрон летит в центр клетки: центр клетки 97 находится в точке $x = 40$ см, $y = 320$ см.

5.3. Четверть

В классе такой большой коврик обычно не разложишь. Коврик складывают в четыре, и получается квадрат 6×6 клеток, то есть 2,4 м на 2,4 м. Это **четверть**. Большинство примеров в книге рассчитано на одну четверть.

Номера клеток всегда считаются по всему коврику. В левой нижней четверти это клетки со столбцами 0–5 и строками 0–5 (рисунок 5.2). Вот их номера по строкам: 0–5, 12–17, 24–29, 36–41, 48–53, 60–65.

5.4. Метки на коврике

Метки на коврике называются AprilTag: это чёрно-белые квадраты, похожие на упрощённые QR-коды. Дрон узнаёт метку камерой и определяет по ней своё место. Позиция у дрона появляется не сразу после включения, а в полёте, примерно на высоте 30 см. Точность около ± 10–15 см: поэтому в шоу не стоит планировать проходы с зазором меньше этого.

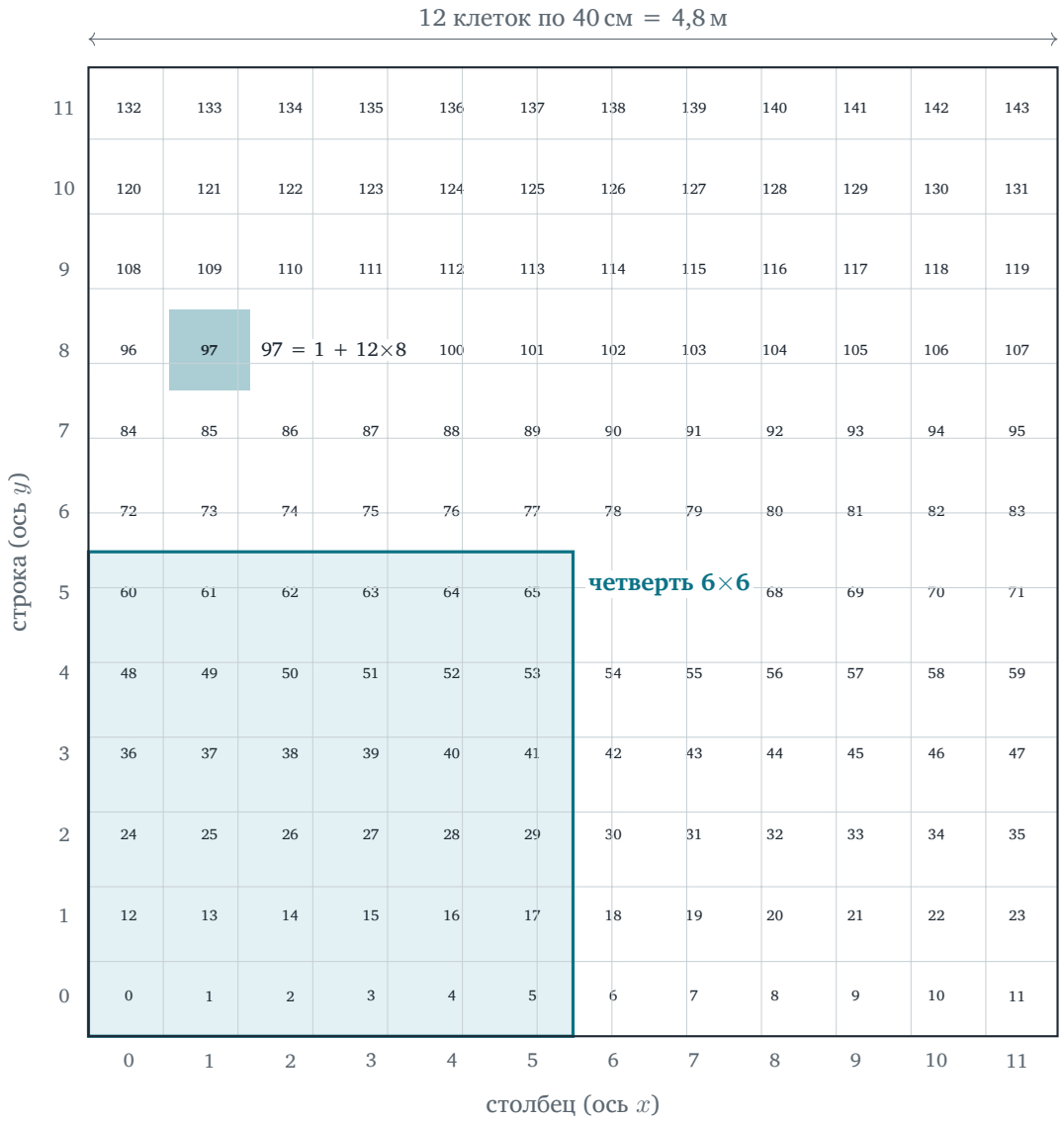


Рис. 5.1.. Коврик 12×12. Клетка 0 внизу слева, номера растут вправо и вверх. Рамкой выделена четверть 6×6.

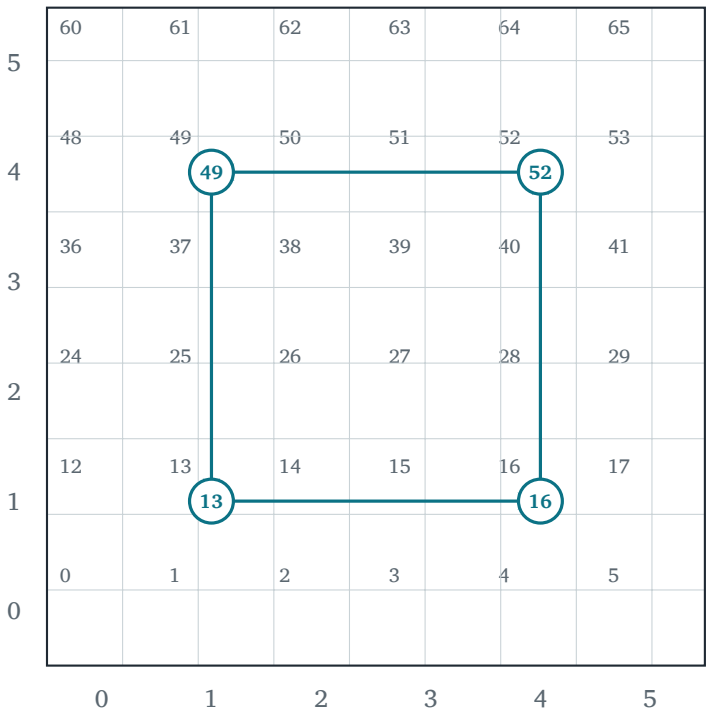


Рис. 5.2.. Левая нижняя четверть и маршрут 13, 16, 52, 49, 13: квадрат со стороной в три клетки.

Попробуйте сами

5.1 Какой номер у клетки в столбце 3 и строке 2?

5.2 В каком столбце и в какой строке клетка 40?

5.3 Чему равны x и y центра клетки 52?

5.4 Сколько сантиметров между центрами клеток 13 и 16? А между 13 и 52?

5.5 Сколько клеток в четверти? Какой номер у её правой верхней клетки?

5.6 Какой номер у клетки прямо над клеткой 97?

Ответы к упражнениям этого урока на странице 65.

Урок 6

Перелёт в клетку

Что вы узнете: как послать дрона в клетку и как связаны расстояние, время и скорость.

6.1. Команда code

`d.code(cell, seconds)` посылает дрона в центр клетки `cell` по прямой за `seconds` секунд.

solutions/l06_move.py

```
# Из клетки 13 в клетку 16: три клетки вправо, 120 см за 4 секунды.
from litebee_show import Show

show = Show('Перелёт')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 4)
d.land()
```

После взлёта идёт `d.code(13, 1)`. Это *стартовая клетка*: дрон должен стоять на клетке 13 и узнать там своё место, а симулятор считает, что дрон начинает под первой клеткой своей программы. Потом дрон перелетает в клетку 16: три клетки вправо, то есть 120 см.

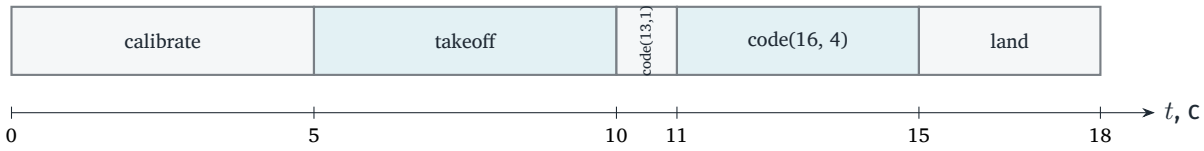


Рис. 6.1.. Лента шоу `l06_move.py`: $5 + 5 + 1 + 4 + 3 = 18$ секунд.

> **Поставить дрона на другую клетку**
>
> Стартовую клетку можно поменять и не набирая число. Перетащите дрона мышью на другую клетку (и на карте сверху, и в 3D). Или щёлкните по дрону, на вкладке «Код» нажмите «Выбрать клетку» и щёлкните по клетке на карте. Приложение само поправит число в первой строке `code` программы; отменить можно Ctrl + Z в редакторе.

6.2. Скорость выбираете вы

Скорость дрона вы не задаёте. Она получается из расстояния и секунд:

$$\text{скорость} = \frac{\text{расстояние}}{\text{секунды}}.$$

Расстояние от клетки 13 до клетки 16 равно $3 \times 40 = 120$ см. За 4 секунды получается $120/4 = 30$ см/с. Если написать `d.code(16, 1)`, скорость будет 120 см/с, и проверка скажет:

«Дрон 1: переход на клетку 16 за 1 с (120 см, 120 см/с) быстрее порога 100 см/с: дрон может перелететь клетку»

Порог такой: **не быстрее 100 см/с**. Если превысить, настоящий дрон может проскочить мимо клетки.

Запомните

Расстояние считается по прямой между центрами клеток. По диагонали оно длиннее: от клетки 13 до клетки 52 это $\sqrt{120^2 + 120^2} \approx 170$ см, значит нужно не меньше 1,7 секунды.

Попробуйте сами

6.1 Перелетите из клетки 13 в клетку 15 за 3 секунды. Чему равна скорость?

6.2 Файл `bad_examples/bad_fast.py` (внизу) получает замечание. Исправьте его.

6.3 Туда и обратно: из клетки 13 в клетку 17 и обратно, по 4 секунды в каждую сторону. Сколько секунд длится шоу?

6.4 Перелетите из клетки 13 в клетку 52 по диагонали. За какое самое короткое время можно, чтобы не было замечания?

`solutions/bad_examples/bad_fast.py`

```
from litebee_show import Show

show = Show('Слишком быстро')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 1)
d.land()
```

Ответы к упражнениям этого урока на странице 65.

Урок 7

Маршрут из нескольких клеток

Что вы узнете: как пролететь несколько клеток подряд и нарисовать фигуру.

7.1. Квадрат

Чтобы нарисовать квадрат, дрону нужно побывать в четырёх клетках. Можно написать четыре `code`, а можно одну команду `code_s`: в ней клетки перечисляются через пробел в строке, а время делится между ними поровну.

solutions/l07_square.py

```
# Квадрат со стороной 3 клетки: каждая сторона 4 секунды.
from litebee_show import Show

show = Show('Квадрат')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code_s('16 52 49 13', 16)
d.land()
```

Квадрат 3×3 клетки (рисунок 5.2): дрон взлетает в клетке 13, потом летит в клетки 16, 52, 49 и снова в 13. Каждая сторона 120 см. Времени на четыре перехода 16 секунд, значит по 4 секунды на сторону, скорость 30 см/с.

> **Запомните**
>
> Клетки в `code_s` можно записывать списком `[16, 52, 49, 13]` или строкой `'16 52 49 13'`. Общее время делится поровну: на каждую клетку `seconds / число клеток`.

> **Направление**
>
> Если смотреть сверху (клетка 0 внизу слева, строки растут вверх), маршрут 13, 16, 52, 49 идёт *против* часовой стрелки: сначала вправо, потом вверх, потом влево.

7.2. Как выбирать время

- Измерьте расстояние: считайте клетки (одна клетка 40 см) или по диагонали — длиннее.
- Разделите на скорость. Для спокойного полёта берите 25–40 см/с.
- Округлите вверх до целых секунд.

Попробуйте сами

7.1 Треугольник: из клетки 13 в клетки 16, 52 и обратно в 13. Подберите время на каждую сторону. Сколько секунд длится шоу?

7.2 Прямоугольник: 3 клетки в ширину, 2 в высоту (клетки 13, 16, 40, 37).

7.3 Буква Z: сверху вправо (клетки 49, 52), по диагонали вниз (13), потом вправо (16).

7.4 Квадрат в обратную сторону: 13, 49, 52, 16, 13.

Ответы к упражнениям этого урока на странице 67.

Урок 8

Посадка в клетку

Что вы узнете: почему дрон садится не точно в центр клетки и как это исправить командой `land_on`.

8.1. land и land_on

Команда `d.land()` сажает дрона там, где он оказался. Но позиция дрона точна лишь до 10–15 см, и после посадки он оказывается рядом с клеткой, а не в её центре. Если вы хотите, чтобы дрон вернулся ровно на свою клетку (чтобы потом снова стартовать с неё), используйте `d.land_on(cell)`.

solutions/l08_landon.py

```
# Облёт квадрата и посадка в центр стартовой клетки.
from litebee_show import Show

show = Show('Посадка в клетку')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code_s('16 52 49 13', 16)
d.land_on(13)
```

Дрон облетает квадрат и садится в центр клетки 13.

8.2. Что делает land_on

Команда `land_on(cell, seconds=2, low=35)` — это четыре шага подряд:

1. `code(cell, seconds)`: подлететь к клетке (по умолчанию 2 с);
2. `set_z(z, 1.5)`: повисеть 1,5 с, чтобы успокоиться;
3. `set_z(low, 2.5)`: плавно снизиться до `low` (по умолчанию 35 см) за 2,5 с;
4. `land()`: сесть (3 с).

Здесь `z` — высота, на которой дрон был до этого. Если взлёта в программе ещё не было, будет ошибка «land_on: сначала нужен взлёт».

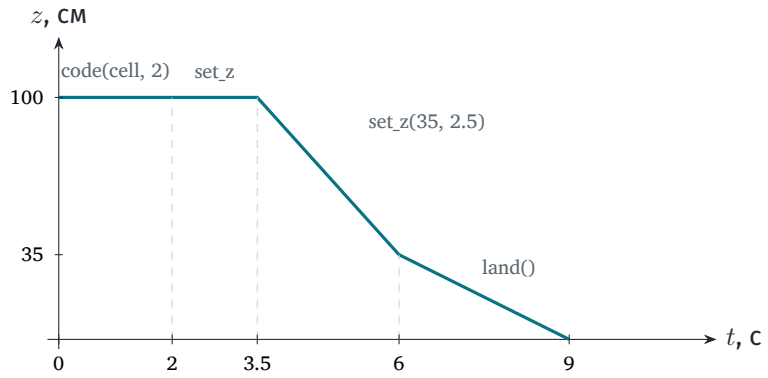


Рис. 8.1.. Высота дрона при `land_on(cell)` с 100 см: подлёт 2 с, зависание 1,5 с, снижение до 35 см за 2,5 с, посадка 3 с. Всего 9 секунд.

Запомните

`land_on` стоит использовать, когда дрон должен вернуться на свою клетку или сесть в заданную. Он делает шоу на 6 секунд длиннее, чем простое `land()` (9 вместо 3).

Попробуйте сами

8.1 Из клетки 13 в клетку 16 и посадка в центр клетки 16.

8.2 То же, но `land_on(16, 3, 60)`: подлёт 3 секунды, остановка на 60 см. Как изменится длительность шоу?

8.3 Замените `land_on(16)` четырьмя обычными командами с теми же временами (из списка выше).

8.4 Из клетки 13 в клетку 52 по диагонали и посадка в центр клетки 52.

Ответы к упражнениям этого урока на странице 68.

Урок 9

Смотрим шоу: партитура

Что вы узнаете: как по партитуре понять, что делает каждый дрон в любую секунду и какая команда за это отвечает.

9.1. Время одно на всё окно

Над вкладками справа всегда видно управление временем: «Играть», скорость, число вида «20,0 / 30,0 с» и ползунок. Время одно на всё окно: на карту, партитуру и код. Передвиньте ползунок, и везде покажется один и тот же момент. Для этого урока возьмите файл, который мы уже знаем:

solutions/l07_square.py

```
# Квадрат со стороной 3 клетки: каждая сторона 4 секунды.
from litebee_show import Show

show = Show('Квадрат')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code_s('16 52 49 13', 16)
d.land()
```

Сложим времена шагов подряд: калибровка 0–5 с, взлёт 5–10, переход в клетку 13: 10–11, квадрат 11–27, посадка 27–30. Всего 30 секунд.

9.2. Партитура

Партитура — это вкладка справа. Она показывает время сверху вниз, а каждому дрону отвечает своя *колонка*.

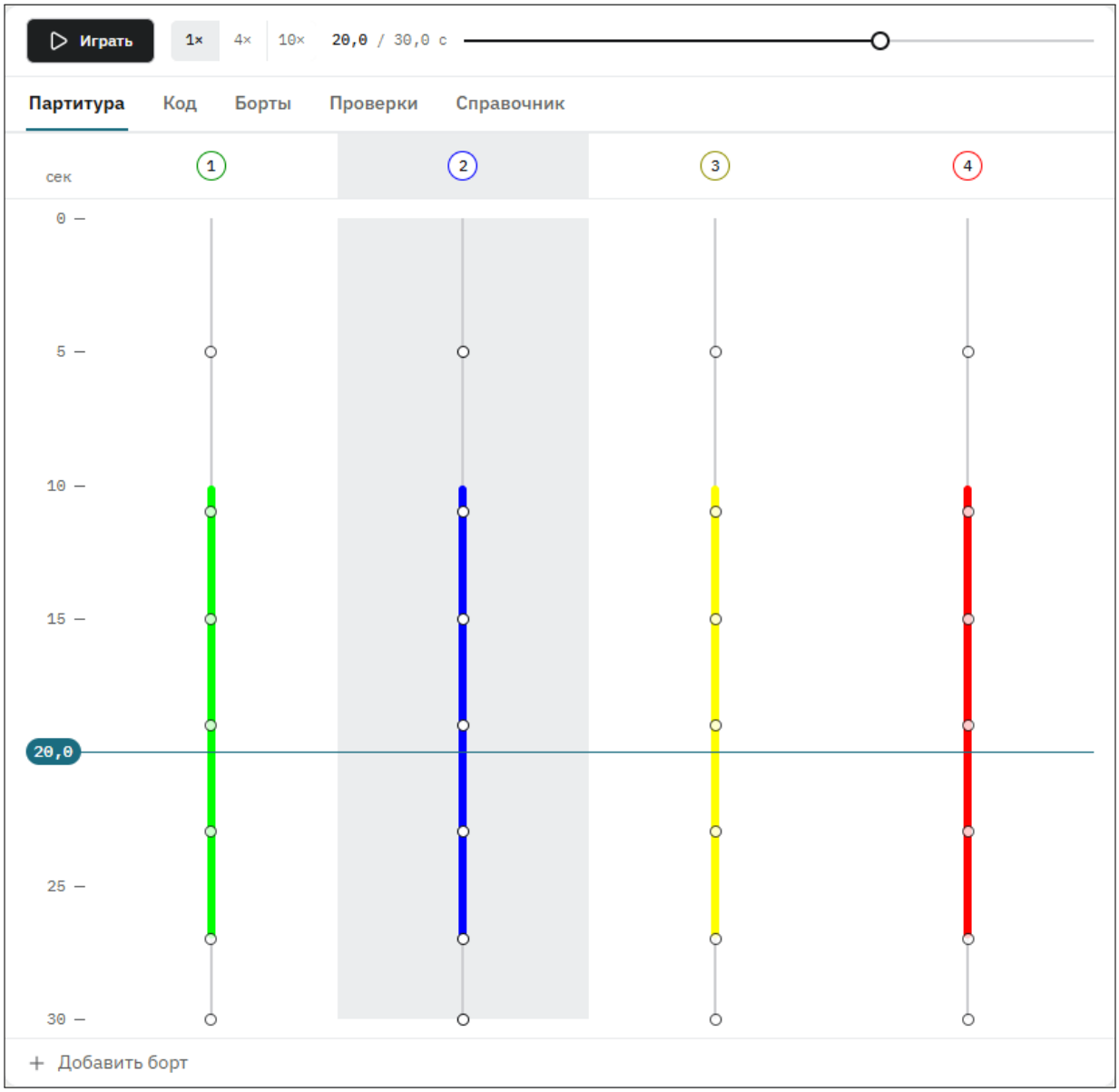


Рис. 9.1.. Партитура шоу «Квадрат». Четыре колонки (дроны 1–4), время идёт сверху вниз. Линия цвета светодиода, кружки-станции на границах шагов, горизонтальная циановая линия с плашкой «20,0» — текущее время. Выбранный дрон 2 выделен серым фоном.

- **Колонка** — один дрон. Вверху кружок с его номером: щелчок выбирает дрона.
- **Цветная линия** показывает цвет светодиода во времени. Когда светодиод выключен, линия серая и тонкая.
- **Кружки-станции** — границы шагов. Между кружками дрон выполняет один шаг.
- **Циановая линия с плашкой** — текущее время.
- **Шкала слева** в секундах, чёрточка каждые 5 секунд.

Если навести мышь на колонку, появится подсказка вида «Дрон 4, 7,4 с: взлёт на 100 см, строка 47»: что делает дрон в этот момент и в какой строке это записано.

9.3. Как двигаться по времени

- Мышью: щёлкните по колонке или тяните по ней вверх и вниз (ползунок сверху работает так же).
- Стрелками: нажмите на партитуру, потом ↑ и ↓ двигают время на секунду, с Shift на десять секунд.
- ← и → выбирают соседнего дрона.

Внизу партитуры есть кнопка «Добавить борт»: она добавляет в шоу нового дрона.

9.4. Выбранный дрон и его код

Когда вы выбираете дрона (на карте или в партитуре), на вкладке «Код» открывается его программа. В коде подсвечена команда, которая выполняется в выбранный момент. Над кодом написано, на какой клетке стоит дрон, а на карте он тоже подсвечен. Так легко найти, какая строка отвечает за то, что вы видите.

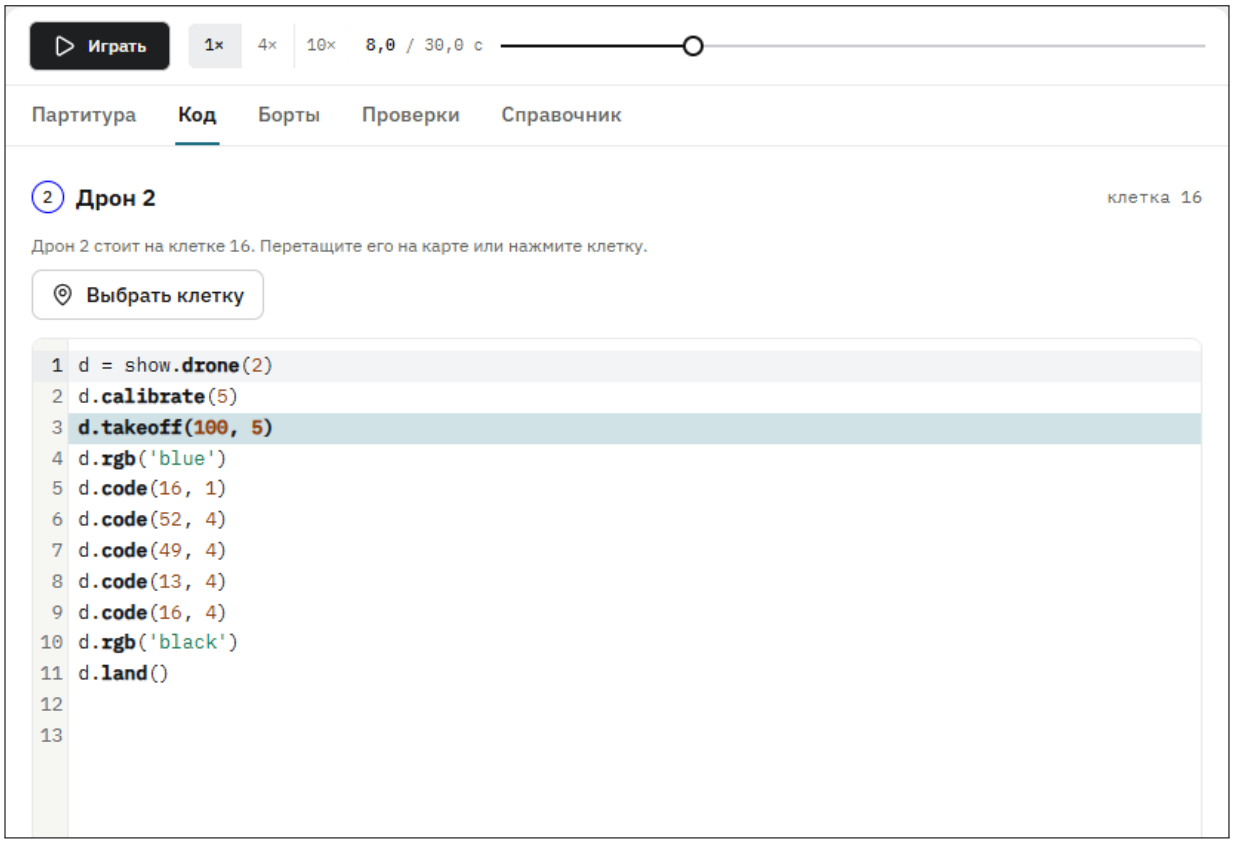


Рис. 9.2.. Вкладка «Код». Выбран дрон 2: клетка 16, кнопка «Выбрать клетку» и программа. Подсвеченная строка (`takeoff`) идёт на 8-й секунде. Номера строк здесь свои у каждого дрона: первая строка — `d = show.drone(2)`.

> **Запомните**
>
> Номера строк в программе дрона начинаются с единицы у каждого дрона. В листингах этой книги номера строк другие: там напечатан весь файл, с комментарием и первыми строками.

> Поэтому в упражнениях ниже мы спрашиваем про команды, а не про номера строк.

Попробуйте сами

9.1 Какая команда файла `l07_square.py` идёт на 8-й секунде?

9.2 А на 20-й секунде?

9.3 На какой секунде дрон приходит в клетку 52?

9.4 Сколько секунд длится шоу?

9.5 Откройте встроенный пример «Разминуться». Выберите дрона 2 и найдите на партитуре его взлёт. Какого цвета у него светодиод?

Ответы к упражнениям этого урока на странице 69.

Урок 10

Два дрона

Что вы узнете: как написать шоу для двух дронов, не столкнув их.

10.1. Два блока

Для второго дрона просто добавьте ещё один блок `d = show.drone(2)` и его команды. Проще нажать «Добавить борт» внизу партитуры: приложение само допишет программу нового дрона и поставит его на свободную клетку внутри показанной области, не ближе двух клеток от остальных. Каждый дрон выполняет свою программу, они не знают друг о друге. Все дроны стартуют одновременно.

solutions/l10_pass.py

```
# Два дрона меняются местами на одной линии: один на 80 см, другой на 150 см.
from litebee_show import Show

show = Show('Разминуться')

d = show.drone(1)
d.calibrate(5)
d.takeoff(80, 5)
d.rgb('blue')
d.code(13, 1)
d.code(16, 6)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(150, 5)
d.rgb('red')
d.code(16, 1)
d.code(13, 6)
d.land()
```

Дроны стоят в клетках 13 и 16 и меняются местами. Они летят по одной линии навстречу друг другу, поэтому один летит на 80 см, а другой на 150 см.

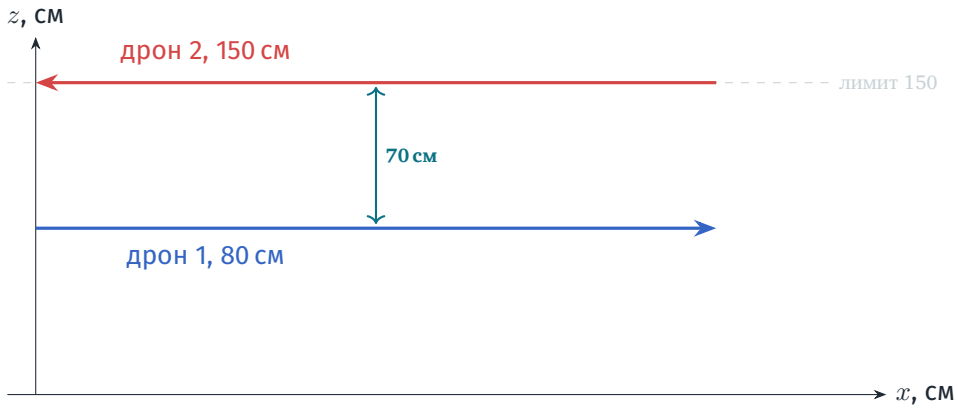


Рис. 10.1.. Вид сбоку: дроны проходят друг над другом с зазором 70 см.

10.2. Правила безопасной дистанции

Расстояние между дронами считается в трёх измерениях, вместе с высотой. Проверка требует, чтобы оно **всегда было не меньше 60 см**.

- **Старт.** Дроны стоят на земле: стартовые клетки соседних дронов держите не ближе двух клеток (80 см).
- **Полёт.** Если пути пересекаются, разведите дронов по высоте не меньше чем на 60 см; лучше 70 и больше.
- **Посадка.** Конечные клетки тоже не ближе двух клеток.

Вот что будет, если оба дрона летят на одной высоте:

solutions/bad_examples/bad_close.py

```
from litebee_show import Show

show = Show('Слишком близко')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 4)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.code(15, 1)
d.code(12, 4)
d.land()
```

Проверка скажет: «Дроны 1 и 2 сближаются до 2 см на 12,3 с (порог 60 см)». Можно передвинуть время на 12,3 с и посмотреть на карте, как дроны проходят друг через друга.

Запомните

Чтобы развести дронов, есть три способа: разные высоты, разное время (один ждёт, пока пролетит другой) и разные маршруты. Первые два приёма в этой книге: высоты здесь, ожидание в уроке 12.

Попробуйте сами

10.1 Два дрона летят рядом, не пересекаясь: 13 → 49 и 16 → 52, оба на 100 см. Почему здесь не нужны разные высоты?

10.2 «Крест»: дрон 1 летит из клетки 13 в 52, дрон 2 из 16 в 49. Они пересекаются в центре. Разведите их по высоте (80 и 150 см).

10.3 Дрон 1 облетает квадрат (клетки 13, 16, 52, 49), а дрон 2 висит выше в клетке 27 и мигает красным четыре раза. Подберите высоты, чтобы проверка прошла.

10.4 Исправьте файл `bad_close.py` выше.

Ответы к упражнениям этого урока на странице 70.

Урок 11

Проверки

Что вы узнете: что ловит кнопка «Проверить», как читать список замечаний и как их исправлять.

11.1. Что проверяет симулятор

Кнопка «Проверить» запускает шоу в симуляции, а список появляется на вкладке «Проверки». Каждая строка бывает одного из трёх уровней.

- **Ошибка**: так лететь нельзя. Кнопка «Загрузить» недоступна.
- **Замечание** (предупреждение): лететь можно, но стоит подумать.
- **Сведение**: просто полезная информация.

Наверху списка видны числа: «ошибок 0, замечаний 1, сведений 1». Ошибки всегда идут первыми.

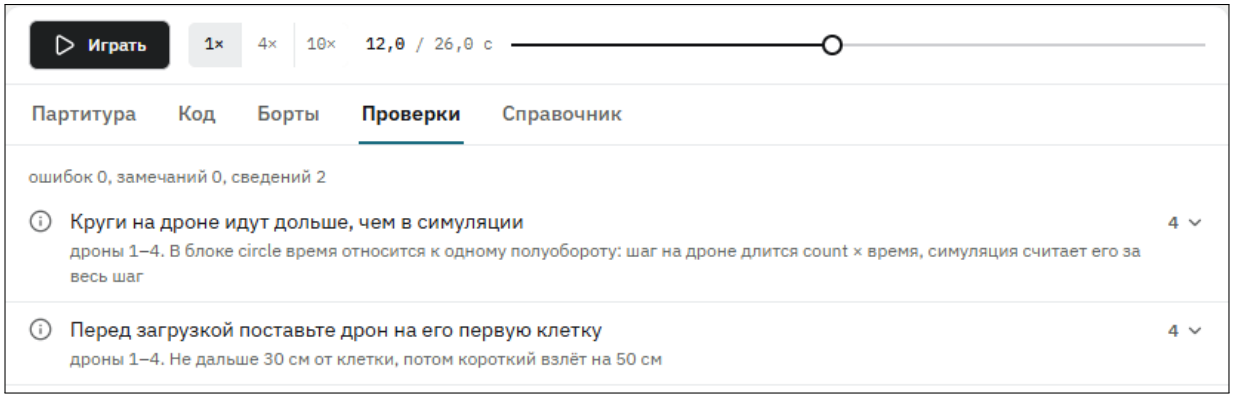


Рис. 11.1. Вкладка «Проверки» для шоу «Круг»: два сведения (круги на дроне идут дольше, перед загрузкой поставьте дрон на первую клетку). Ошибок нет. Строку можно раскрыть стрелкой справа.

Всего проверяется пять вещей.

Что	Правило	Уровень
Область	дрон не выходит за выбранную область; клетка не больше 143	ошибка
Высота	не выше лимита (по умолчанию 150 см)	ошибка
Скорость	каждый переход не быстрее 100 см/с	замечание
Сближение	дроны не ближе 60 см друг к другу	замечание
Зависание	одно ожидание не длиннее 60 с	отказ при загрузке

Про скорость и сближение вы уже знаете (уроки 6, 10). Остальное разберём на примерах.

11.2. Читаем сообщение

В каждом сообщении три вещи:

1. **кто**: номер дрона или пара дронов;
2. **когда**: время в шоу; передвиньте ползунок на это время и посмотрите;
3. **где**: номер строки в коде (щелчок по сообщению показывает её).

Похожие замечания о разных дронах приложение собирает в одну строку, например «Переход быстрее порога… дроны 1–10, быстрейший 120 см/с». Строку можно раскрыть стрелкой.

11.3. Три проблемы сразу

В этом файле нарочно собрано сразу три проблемы:

solutions/bad_examples/bad_three.py

```
from litebee_show import Show

show = Show('Три проблемы')

d = show.drone(1)
d.calibrate(5)
d.takeoff(170, 5)
d.code(13, 1)
d.code(16, 1)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(170, 5)
d.code(15, 1)
d.code(13, 6)
d.land()
```

Проверка скажет:

- ошибка: «Дрон 1: высота 170 см больше лимита 150 см, полёт будет отклонён» (и то же про дрон 2);
- замечание: «Дрон 1: переход на клетку 16 за 1 с (120 см, 120 см/с) быстрее порога 100 см/с…»;
- замечание: «Дроны 1 и 2 сближаются до 0 см на 11,6 с (порог 60 см)».

Чтобы все три исчезли, нужно: понизить высоты, добавить времени переходу и развести дронов по высоте.

11.4. Лимит высоты и область

Лимит. В меню «Шоу…» над сценой есть раздел «Высота» с полем «Лимит высоты, см»: оно принимает целые числа от 20 до 200. После смены нажмите «Проверить» ещё раз. Больше 200 нельзя.

Область. Выбор «Область» над сценой задаёт рамку (весь коврик или четверть). Если дрон вылетает за неё, будет ошибка «полёт выходит за рабочую область». Тогда смените область, перенесите шоу (меню «Шоу…») или поправьте клетки. Сам код выбор области не меняет.

Зависание. Если ожидание на месте длиннее 60 секунд, приложение откажет при «Загрузить»: «Зависание 90 с больше лимита 60 с». Разбейте ожидание на несколько шагов.

Попробуйте сами

11.1 Исправьте все три проблемы файла `bad_three.py`, чтобы проверка прошла без замечаний.

11.2 В файле `bad_height.py` (урок 2) высота 180 см. Не меняя файл, сделайте так, чтобы проверка прошла. Что для этого нужно?

11.3 В файле `bad_hover.py` дрон висит 90 секунд на одном месте. Исправьте его.

11.4 В файле `bad_area.py` дрон летит в клетку 8, а выбрана четверть 1. Исправьте файл.

solutions/bad_examples/bad_hover.py

```
from litebee_show import Show

show = Show('Слишком долгое ожидание')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.set_z(100, 90)
d.land()
```

solutions/bad_examples/bad_area.py

```
from litebee_show import Show

show = Show('Мимо четверти')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(8, 4)
d.land()
```

Ответы к упражнениям этого урока на странице 72.

Урок 12

Очередь и ожидание

Что вы узнете: как сделать так, чтобы один дрон начинал после другого.

12.1. У дронов нет общих часов

Помните: у каждого дрона свои часы, и они начинаются с нуля. Все дроны стартуют вместе, но они не знают, что делают другие. Длина шоу — это длительность самого долгого дрона.

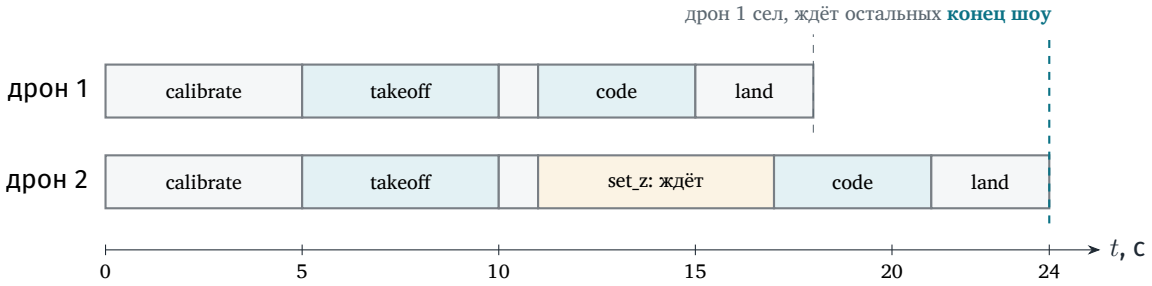


Рис. 12.1.. Ленты двух дронов: первому нужно 18 секунд, второму 24. Шоу длится 24 секунды. Второй ждёт (светлый отрезок), пока первый летит.

Поэтому «сделай это после дрона 1» нужно устроить самому: сложить секунды дрона 1 и вставить ожидание.

12.2. Эстафета

Три дрона по очереди слетают вверх по своему столбцу и назад. Пока летит один, остальные ждут и горят синим.

solutions/l12_relay.py

```python
# Каждый дрон слетает вверх по своему столбцу и назад; остальные ждут.
from litebee_show import Show

show = Show('Эстафета')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.rgb('green')
d.code(49, 4)
d.code(13, 4)
d.rgb('blue')
d.set_z(100, 18)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.code(15, 1)
d.rgb('blue')
d.set_z(100, 9)
d.rgb('green')
d.code(51, 4)
d.code(15, 4)
d.rgb('blue')
d.set_z(100, 9)
d.land()

d = show.drone(3)
d.calibrate(5)
d.takeoff(100, 5)
d.code(17, 1)
d.rgb('blue')
d.set_z(100, 18)
d.rgb('green')
d.code(53, 4)
d.code(17, 4)
d.land()
```

Посчитаем время второго дрона.

- К секунде 11 он закончил калибровку (5), взлёт (5) и выход на свою клетку (1).
- Дрон 1 с 11-й секунды летит $4 + 4 = 8$ секунд, значит освободится на 19-й.
- Дрон 2 ждёт `set_z(100, 9)`, то есть до 20-й секунды. Секунда лишняя, это запас.
- Потом он летит 8 секунд, до 28-й, и дрон 3 ждёт `set_z(100, 18)`, до 29-й.

Запомните

Формула ожидания: *ожидание = сумма времён шагов того, кого ждёшь, + запас 1 секунда.* Если потом поменять время у первого дрона, ожидание придётся пересчитать вручную.

Осторожно

Настоящие дроны стартуют не в один миг и летят не точно по расписанию. Оставляйте запас в секунду-две.

Попробуйте сами

12.1 Два дрона по очереди: сначала дрон 1 летит из клетки 13 в клетку 49 и обратно (по 4 с), потом дрон 2 из 15 в 51 и обратно.

12.2 Три дрона зажигаются красным по очереди на две секунды: сначала первый, потом второй, потом третий. Остальное время они не светят.

12.3 «Волна»: три дрона на 80 см по очереди поднимаются до 150 см и опускаются обратно (по 2 секунды вверх и вниз).

12.4 Дрон 1 летит из 13 в 17 за 6 секунд, дрон 2 из 37 в 41 за 6 секунд, но на 3 секунды позже. Как записать «позже»?

Ответы к упражнениям этого урока на странице 73.

Урок 13

Повторы: циклы

Что вы узнете: как не писать одно и то же много раз.

13.1. Цикл в программе одного дрона

Три красные вспышки можно записать шестью строками, а можно одним циклом:

solutions/l13_blink.py

```
# Три красных вспышки: тело цикла повторяется три раза.
from litebee_show import Show

show = Show('Мигание циклом')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
for _ in range(3):
    d.rgb('red')
    d.set_z(100, 1)
    d.rgb('black')
    d.set_z(100, 1)
d.land()
```

`for _ in range(3):` значит «повтори три раза». Всё, что в следующих строках записано *со сдвигом вправо* (четыре пробела), — тело цикла, оно повторяется. Сдвиг вправо в Python обязателен: так он понимает, где кончается цикл.

Подчёркивание `_` здесь вместо имени счётчика: нам не важен номер повтора. Если номер нужен, берут имя: `for n in range(3):` даст `n` = 0, 1, 2.

13.2. Цикл по списку

`for color in colors:` перебирает значения списка по одному:

```
colors = ['red', 'green', 'blue']
for color in colors:
    d.rgb(color)
    d.set_z(100, 1)
```

13.3. Цикл по дронам

Если много дронов делают похожее, цикл можно поставить *вокруг* `show.drone`:

solutions/t10_loop.py

```
# Проект 3 из урока 16, записанный циклом: один блок на всех четырёх дронов.
from litebee_show import Show

show = Show('Углы циклом')

corners = [0, 4, 52, 48]
colors = ['red', 'yellow', 'green', 'blue']

for n in range(4):
    d = show.drone(n + 1)
    d.calibrate(5)
    d.takeoff(100, 5)
    d.rgb(colors[n])
    d.code(corners[n], 1)
    for k in range(1, 5):
        d.code(corners[(n + k) % 4], 4)
    d.rgb('black')
    d.land()
```

Каждый проход создаёт очередного дрона: `show.drone(n + 1)`. Список `corners` хранит клетки, `colors` цвета. Выражение `corners[(n + k) % 4]` берёт клетку по номеру, а `%` (остаток от деления) возвращает счёт к нулю после четвёртой клетки: после номера 3 идёт 0.

> **Циклы и программа дрона**
>
> Обычно вкладка «Код» показывает программу одного дрона. Если же дроны создаёт общий цикл, отдельной программы у каждого нет, и приложение показывает код целиком: «Дроны этого шоу создаёт общий цикл, поэтому код показан целиком». Это не ошибка: симуляция, партитура и проверки работают как обычно. Цикл *внутри* программы одного дрона такому показу не мешает.

Попробуйте сами

13.1 «Челнок»: три раза туда и обратно между клетками 13 и 17 (по 4 с), циклом.

13.2 Радуга: список из семи цветов `['red', 'yellow', 'green', 'cyan', 'blue', 'purple', 'white']` и цикл по нему, по секунде каждый.

13.3 Три дрона в клетках 13, 15, 17 мигают красным по очереди (по 2 секунды). Пауза зависит от номера дрона. Запишите циклом по дронам.

13.4 Квадрат три раза подряд: цикл `for lap in range(3):` с `code_s`.

Ответы к упражнениям этого урока на странице 76.

Урок 14

Другая четверть

Что вы узнете: как перенести готовое шоу в другую часть коврика.

14.1. Сдвиг номеров

Номера клеток всегда считаются по всему коврику, а четверть — окно на его часть. Чтобы то же шоу полетело в другой четверти, нужно сдвинуть номера клеток. Левая нижняя клетка каждой четверти:

Четверть	Где	Сдвиг номеров
1	слева внизу	0
2	справа внизу	+6
3	слева вверху	+72
4	справа вверху	+78



Рис. 14.1.. Четыре четверти коврика и сдвиг номеров клеток.

Вот квадрат из урока 7, перенесённый во вторую четверть. Ко всем клеткам прибавлено 6: 13 → 19, 16 → 22, 52 → 58, 49 → 55.

solutions/l14_quarter2.py

```
# Квадрат из урока 7, к каждой клетке добавлено 6.
from litebee_show import Show

show = Show('Квадрат во второй четверти')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(19, 1)
d.code_s('22 58 55 19', 16)
d.land()
```

14.2. Перенос в меню «Шоу...»

Складывать вручную не обязательно. В меню «Шоу...» над сценой есть раздел «Перенести шоу в четверть»: выберите четверть, подтвердите «Сдвинуть шоу в четверть N? Номера клеток в коде

изменятся», и номера клеток сдвинутся сами. Отменить можно клавишами `Ctrl` + `Z` в редакторе.

> **Область и перенос**
>
> Выбор области сам код не меняет, это только рамка для проверки. Если шоу написано для клеток 0–5, а область выбрана другая, проверка скажет «полёт выходит за рабочую область». Тогда смените область или перенесите шоу.

Попробуйте сами

14.1 Перенесите квадрат из урока 7 в третью четверть (прибавьте 72).

14.2 Перенесите тот же квадрат в четвёртую четверть (прибавьте 78).

14.3 Чему равна клетка 14 в третьей четверти? А клетка 55 в четвёртой?

14.4 Клетка 100: в какой она четверти? Какой номер у клетки на том же месте в четверти 1?

Ответы к упражнениям этого урока на странице 77.

Урок 15

Круги и подъём на ходу

Что вы узнете: как нарисовать круг и как лететь вверх и вперёд одновременно.

> **Сначала простые команды**
>
> Шаг `circle` уже летал на живом дроне, но время круга на дроне и на экране разное (см. ниже). Шаг `code_z` на живом дроне **ещё не летал**: приложение упаковывает его так же, как LiteBee Go, а в списке проверок вы увидите замечание «Шаги `code_z` ещё не летали на живом дроне». Для первых полётов пользуйтесь командами из уроков 2–8.

15.1. Круг: circle

`d.circle(count, cell1, cell2, seconds, rotate='cw')` — круг вокруг *середины* между клетками `cell1` и `cell2`.

- `count` — число *полуоборотов*: 2 полукруга — полный круг, дрон вернётся на место.
- Центр — середина между клетками. Если обе клетки одинаковые, центр — её центр.
- Радиус — расстояние от дрона до центра. Дрон крутится вокруг центра, проходя через то место, где стоял.
- `rotate`: `'cw'` (по часовой) или `'ccw'` (против).

solutions/l15_circle.py

```
# Центр круга в клетке 26, дрон стартует в клетке 14, радиус 40 см.
from litebee_show import Show

show = Show('Круг')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(14, 1)
d.circle(2, 26, 26, 12)
d.land()
```

Дрон стоит в клетке 14, центр круга — центр клетки 26. Радиус 40 см.

> **Время круга**
>
> В симуляторе число секунд — это время всего круга. На живом дроне иначе: секунды — время одного полуоборота, и шаг длится `count` раз дольше. Круг на дроне получится дольше, чем на экране. Куда на коврике поворачивает `'cw'`, в симуляторе угадано.

15.2. Подъём на ходу: code_z

`d.code_z(cell, z, seconds)` летит в клетку и одновременно меняет высоту. Число `z` — *итоговая* высота, а не прирост.

solutions/l15_climb.py

```
# Из 13 в 49 с подъёмом на 150 см и обратно с опусканием на 60.
from litebee_show import Show

show = Show('Подъём на ходу')

d = show.drone(1)
d.calibrate(5)
d.takeoff(60, 5)
d.code(13, 1)
d.code_z(49, 150, 6)
d.code_z(13, 60, 6)
d.land()
```

Дрон со 60 см поднимается до 150 см, пока летит из клетки 13 в клетку 49, и возвращается к 60 см по пути назад. Расстояние теперь считается по трём осям, скорость тоже.

Есть ещё `circle_z` (полкруга с подъёмом), но его **нельзя загрузить в дрон**, поэтому в учебных шоу он не используется.

Попробуйте сами

15.1 Тот же круг в другую сторону (`'ccw'`).

15.2 Половина круга (`count = 1`): дрон оказывается напротив своего места. Где именно, если он стартует в клетке 14 с центром 26? Посадите его через `land_on`.

15.3 «Горка»: со 60 см до 140 см по пути из 13 в 16 и обратно.

15.4 Два дрона в клетках 14 и 38 крутятся вокруг клетки 26, один напротив другого. Почему они не столкнутся?

Ответы к упражнениям этого урока на странице 78.

Урок 16

Итоговые проекты

Что вы сделаете: соберёте всё вместе: от двух дронов до восьми. Задачи идут по нарастающей. Общий критерий для всех: **«Проверить» не показывает ни ошибок, ни замечаний** (сведения допускаются). Дроны летают в левой нижней четверти, клетки 0–5 по столбцам и строкам.

> **Общие правила**
>
> - Стартовые клетки соседних дронов не ближе двух клеток (80 см).
> - Каждый дрон начинает с `calibrate(5)` и `takeoff(...)`, потом первая `code` указывает его стартовую клетку.
> - Расстояние между дронами не меньше 60 см в любой момент. Если пути пересекаются, разводите дронов по высоте или по времени.
> - Скорость перехода не больше 100 см/с, высота не больше 150 см.
> - Заканчивайте программу посадкой.

16.1. Проект 1. За лидером (2 дрона)

Дрон 1 (80 см) облетает квадрат 13, 16, 52, 49 и возвращается в клетку 13. Дрон 2 (150 см) стартует из клетки 37 и повторяет тот же маршрут на один переход (4 секунды) позже, а в конце садится в клетку 37.

Подсказка. Дрон 2 сразу выходит в клетку 13 за 4 секунды: пока лидер идёт в клетку 16, он занимает его место. Тогда он всё время на один шаг позади.

16.2. Проект 2. Лесенка (3 дрона)

Три дрона в клетках 13, 15 и 17 на разных высотах сдвигаются по кругу: 1 на место 2, 2 на место 3, 3 на место 1.

Подсказка. Дрон, летящий в самый дальний угол, пройдёт над двумя другими. Подберите три высоты от 20 до 150 см с зазором не меньше 60 см.

16.3. Проект 3. Четыре угла (4 дрона)

Четыре дрона стоят по углам квадрата 5×5 клеток (клетки 0, 4, 52, 48) и обходят его по кругу: каждый переходит в следующий угол и делает полный оборот.

Подсказка. Все дроны движутся одновременно в одну сторону, на разные высоты разводить не нужно. Сторона 160 см, хватит 4 секунд на переход. Вариант с циклом лежит в файле `t10_loop.py`.

16.4. Проект 4. Пятёрка (5 дронов)

Четыре дрона по углам квадрата идут по кругу на два угла вперёд, а пятый в центре (клетка 26) поднимается на 120 см и опускается на 60 см.

Подсказка. Центр отстоит от угловых на 113 см, а в середине стороны угловые проходят в 80 см от него: этого хватает без разведения по высоте.

16.5. Проект 5. Ряды меняются (6 дронов)

Шесть дронов стоят в двух рядах по три: нижний ряд в клетках 13, 15, 17, верхний в клетках 49, 51, 53. Ряды меняются местами. Нижний ряд летит на 80 см, верхний на 150 см.

Подсказка. Дроны одного столбца летят навстречу друг другу, поэтому им нужны разные высоты. Соседние столбцы разделены 80 см.

16.6. Проект 6. Семёрка (7 дронов)

Шесть дронов стоят шестиугольником вокруг клетки 26 (клетки 2, 16, 40, 50, 36, 12), седьмой в центре. Шестёрка по кольцу сдвигается на два места, а центральный мигает белым пять раз.

Подсказка. Соседние дроны кольца на высотах 90 и 130 см, чтобы разойтись при сдвиге. Центральный висит ниже: 60 см.

16.7. Проект 7. Хоровод (8 дронов)

Восемь дронов стоят по кольцу в клетках 0, 2, 4, 28, 52, 50, 48, 24. Дроны с чётными номерами (на 150 см) идут по кольцу в одну сторону, с нечётными (на 80 см) навстречу. Каждый проходит два места и садится в центр клетки.

Подсказка. Соседние дроны на разных высотах, поэтому встречные разминутся. Кольцо задайте списком и берите клетки по индексу с остатком от деления на 8.

Ответы к упражнениям этого урока на странице 79.

Урок 17

От симулятора к дрону

Этот урок короткий и осторожный. Настоящий дрон весит, крутит винты и может ударить. Всё, что ниже, делайте только вместе с учителем или взрослым и только после того, как шоу прошло проверку на экране.

> **Правила первого полёта**
>
> - Первый полёт делайте **одним дроном**, а не всем строем.
> - Рядом с ковриком не должно быть людей. Те, кто наблюдает, стоят за границей зоны.
> - Рука лежит на красной кнопке «Аварийная посадка» или на клавише `Esc`.
> - Шаг `code_z` на живом дроне **ещё не летал**, а у `circle` время на дроне другое (см. ниже). Если они есть в шоу, будьте особенно осторожны.

17.1. Цепочка перед полётом

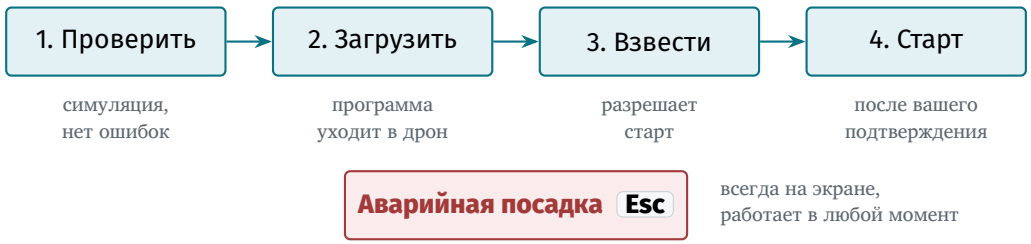


Рис. 17.1.. Четыре шага перед полётом и красная кнопка, которая работает всегда.

1. **Проверить.** Нет ошибок и нет неожиданных замечаний.
2. **Загрузить.** Программа каждого дрона превращается в набор шагов и уходит в выбранные дроны. Приложение откажет, если высота выше лимита, ожидание дольше 60 с, батарея ниже 7,0 В или дрон стоит не на своём месте. Подсказка рядом с кнопкой объясняет причину.
3. **Взвести.** Разрешает старт. Взвод можно снять ещё одним щелчком.
4. **Старт.** Появляется вопрос с подтверждением. Только после него дроны полетят.

Цепочка сама подсказывает следующую кнопку. Если дронов на связи нет, шоу можно писать, смотреть и проверять, но не лететь: кнопки загрузки и старта остаются серыми, и подсказка объясняет причину.

17.2. Связь и дроны

Дроны сами находят компьютер в сети. Связь включается в боковой панели у левого края окна (она выезжает при наведении мыши): выберите сетевой адаптер, подключённый к Wi-Fi дронов

(адрес вида 192.168.13.x), и нажмите «Слушать». Связь работает только в приложении, а не в браузере. Включённые дроны появляются на вкладке «Борты».

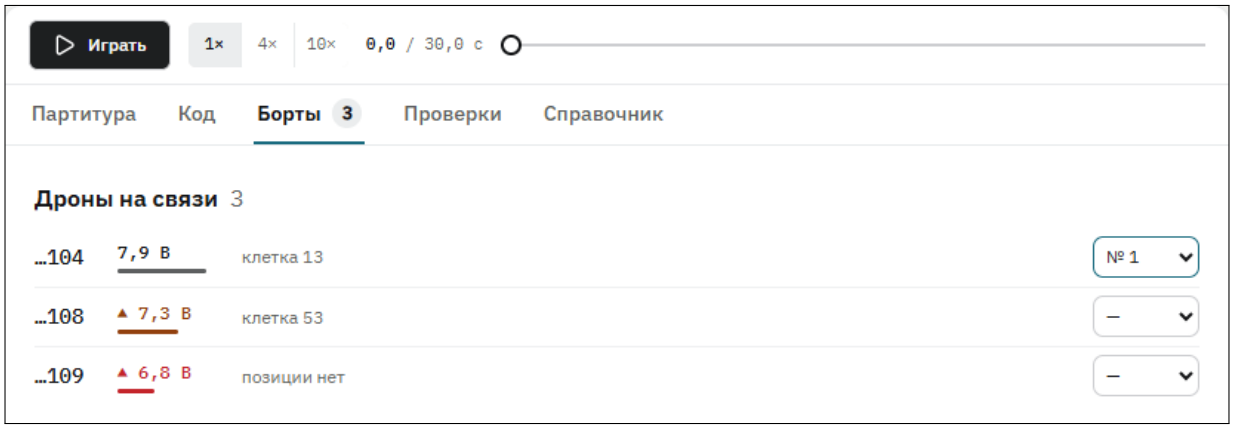


Рис. 17.2.. Вкладка «Борты»: три придуманных дрона на связи. У каждого конец адреса, заряд батареи, клетка коврика и выбор борта шоу («№ 1»). Красная строка в шапке объясняет, почему кнопки загрузки пока нельзя нажимать.

Роли. Приложению нужно знать, какой живой дрон какому борту шоу соответствует. Номер находится сам: если живой дрон стоит на первой клетке борта N, и больше никого на ней нет, приложение выбирает «№ N». Если два дрона стоят на одной клетке, выбор остаётся пустым. Свой выбор главнее автоматического; дрон без выбранного борта в шоу не летит.

17.3. Батарея

- Ниже 7,4 В карточка дрона становится янтарной: лучше заменить батарею.
- Ниже 7,0 В взлёт запрещён: загрузка откажет, порог снизить нельзя.

17.4. Позиция по коврику

Дрон двигается по коврику *абсолютно*: он летит к клетке, откуда бы ни стартовал. Поэтому ему нужна позиция, и стоять он должен рядом с первой клеткой своего движения.

1. Поставьте дрон на коврик на первую клетку, в которую шоу его посылает (какая это клетка, пишет сведение в проверках). Лучше прямо на неё.
2. Сделайте короткий взлёт на 50 см, чтобы дрон получил позицию. Позиция остаётся после посадки.
3. Убедитесь, что в списке дронов появилась клетка и она нужная.
4. Не переносите дрон между загрузкой и стартом: позиция проверяется при старте ещё раз.

Без позиции дрон прервёт миссию с движением по клеткам примерно через 3 секунды, не взлетев. Одна калибровка позицию не даёт.

Две цифры про первую клетку

Проверка допускает расстояние до первой клетки до 200 см, а в подсказке приложение пишет «не дальше 30 см». Берите строгое число: ставьте дрон точно на клетку.

17.5. Что полетит не так, как на экране

- **Точность**: дрон ошибается на 10–15 см, перелёт до 20 см.
- **Посадка**: на экране 3 секунды, на дроне шаг посадки времени не добавляет.
- **Круги**: на дроне шаг `circle` длится в `count` раз дольше.
- **Старт**: симулятор считает, что дрон стоит под первой клеткой на земле. Живой дрон стоит там, где вы его поставили.

17.6. Аварийная посадка

Красная кнопка в правом верхнем углу и `Esc` отправляют дронам команду посадки. Она работает, пока дрон взведён или летит. Если пропала связь или произошла ошибка, приложение само отправляет посадку и пишет в журнал, что случилось. Но не рассчитывайте на это: следите за полётом.

Приложение А

Таблица команд

Все команды дрона. Дрон `d` получен так: `d = show.drone(N)`. Статусы: `проверено` — выполнялось на живом дроне и загружается; `симулятор` — работает на экране, на живом дроне не проверено.

Запись	Что делает	Статус
`calibrate(seconds)`	калибровка перед взлётом, обычно первый шаг	проверено
`takeoff(z, seconds)`	взлёт на высоту `z` см за `seconds` с	проверено
`rgb(color)`	цвет светодиода: имя, номер 0–7 или `(r, g, b)`; занимает 0 с	проверено
`rgb_value(r, g, b)`	цвет по трём каналам 0–255; то же, что `rgb((r, g, b))`	проверено
`code(cell, seconds)`	лететь к клетке по прямой за `seconds` с	проверено
`code_s(cells, seconds)`	пролететь клетки списка или строки `'12 23 27'`, время делится поровну	проверено
`set_z(z, seconds)`	перейти на высоту `z`; на той же высоте — ожидание	проверено
`code_z(cell, z, seconds)`	лететь к клетке и на итоговую высоту `z` сразу; загружается, но не летал	симулятор
`circle(count, cell1, cell2, seconds, rotate='cw')`	круг вокруг середины клеток `cell1` и `cell2`; `count` в полуоборотах; летал на дроне, время иное	проверено
`circle_z(cell1, cell2, dz, seconds, rotate='cw')`	полкруга с подъёмом на `dz` см; в дрон не загружается	симулятор
`land()`	посадка на месте; в симуляции 3 с	проверено
`land_on(cell, seconds=2, low=35)`	посадка в центр клетки: подлёт, зависание 1,5 с, снижение до `low` за 2,5 с, посадка	проверено

Шоу

Запись	Что делает
`Show(name)`	создаёт шоу; переменная должна называться `show`
`show.drone(number)`	возвращает дрона с этим номером, при первом обращении создаёт его

Числа и пределы

Что	Значение
клетка коврика	40 см
коврик	12 × 12, клетки 0–143
четверть	6 × 6, 2,4 × 2,4 м
лимит высоты по умолчанию	150 см (от 20 до 200 см)
наименьшая дистанция между дронами	60 см
наибольшая средняя скорость перехода	100 см/с
посадка в симуляции	3 с
наибольшее ожидание на месте	60 с
батарея: предупреждение / запрет взлёта	7,4 В / 7,0 В
наибольшее число шагов дрона в миссии	255
наибольшее число шагов в шоу	10 000
время запуска кода шоу	5 с

Приложение В

Частые ошибки

Сообщения ниже настоящие: их получили, запустив файлы из `solutions/bad_examples`. Номер в скобках — строка файла, а не шоу.

Ошибки запуска кода

Сообщение	Что случилось и что делать
`Синтаксическая ошибка в строке 6: '(' was never closed`	Не закрыта скобка (`bad_syntax.py`). Посмотрите на указанную строку и строку выше.
`NameError: name 'e' is not defined (строка 7)`	Написано имя, которое не создано, обычно опечатка: `e.land()` вместо `d.land()`.
`ValueError: Неизвестный цвет 'pink'. Доступны: black, red, green, yellow, blue, purple, cyan, white (строка 6)`	Цвета `pink` нет. Возьмите имя из списка, номер 0–7 или тройку `(r, g, b)`.
`ValueError: land_on: сначала нужен взлёт (строка 6)`	`land_on` берёт высоту у последнего `takeoff`, `set_z` или `code_z`. Поставьте их раньше.
`В файле нет переменной show = Show('имя')`	Нет строки `show = Show('имя')` или переменная названа иначе.
`Шоу содержит больше 10000 шагов. Проверьте циклы в коде`	Цикл без конца или с огромным числом повторов (`bad_loop.py`).
`Шоу выполняется дольше 5 с. Проверьте циклы в коде`	Код шоу работает слишком долго; ищите бесконечный цикл.
`Время шага не может быть отрицательным`	Отрицательное число секунд в команде.
`Список клеток пуст`	В `code_s` передан пустой список.
`Номер цвета должен быть от 0 до 7 и Компоненты цвета должны быть от 0 до 255`	Номер цвета или канал вне допустимого диапазона.

Сообщения проверки

Сообщение	Что делать
`Дрон 1: клетка 144 вне коврика 0-143`	Номер клетки может быть от 0 до 143.
`Дрон 1: высота 180 см больше лимита 150 см, полёт будет отклонён`	Уменьшите высоту или поднимите лимит в окошке «Лимит высоты» (не больше 200 см), потом снова «Проверить».

Сообщение	Что делать
`Дрон 1: переход на клетку 16 за 1 с (120 см, 120 см/с) быстрее порога 100 см/с: дрон может перелететь клетку`	Добавьте секунд: расстояние / секунды не больше 100.
`Дроны 1 и 2 сближаются до 2 см на 12.3 с (порог 60 см)`	Разведите дронов по высоте, по времени или по маршруту.
`Зависание 90 с больше лимита 60 с`	Ожидание на месте длиннее 60 с (отказ при загрузке). Разбейте его или проверьте число.
`Дрон N: полёт выходит за рабочую область (…)`	Дрон вылетел за выбранную область. Смените область, нажмите «Перенести шоу» или поправьте клетки.
`Круги на дроне идут дольше, чем в симуляции`	Не ошибка, а предупреждение: на дроне шаг `circle` длится `count` × секунды, на экране секунды за весь круг.
`Шаги code_z ещё не летали на живом дроне`	Не ошибка: шаг в симуляторе работает, на дроне не проверен. Первый полёт делайте одним дроном.
`загрузка в дрон недоступна`	В программе дрона есть шаг, который загрузить нельзя (`circle_z`).
`Перед загрузкой поставьте дрон на его первую клетку`	Сведение: поставьте дрон на первую клетку шоу и сделайте взлёт на 50 см.

Ошибки, которые проверка не видит

- **Забыли ожидание.** Дрон 2 должен начать «после» дрона 1, но у них одновременный старт; вы не вставили `set_z` на той же высоте.
- **Перепутали клетки.** Номер клетки не столбец и не строка, а столбец + 12 × строка. Клетка 16 — это строка 1, а не 6-я клетка.
- **Не тот дрон шоу.** Живой дрон стоит не на первой клетке нужного дрона: загрузка откажет или дрон полетит не туда.
- **Цвет без ожидания.** `rgb` занимает 0 с: если следом идёт ещё один `rgb`, первый цвет не успеет показаться.
- **Два дрона на одной клетке.** В симуляторе дроны стартуют под своей первой клеткой; два дрона на одной клетке дадут замечание о сближении.

Приложение C

Глоссарий

AprilTag
чёрно-белая метка на коврике; камера дрона узнаёт её и определяет позицию.

Аварийная посадка
красная кнопка внизу окна и клавиша `Esc`: дроны на связи получают команду посадки.

Взвести
третий шаг цепочки; разрешает старт.

Борт
дрон шоу с номером N (в коде `show.drone(N)`).

Борты на связи
настоящие дроны, которые приложение видит по Wi-Fi; их показывает вкладка «Борты».

Высота
расстояние от пола до дрона в сантиметрах. Лимит по умолчанию 150 см.

Дрон шоу
дрон из файла шоу с номером `N`; живому дрону соответствует по выбору или по клетке, на которой он стоит.

Загрузить
второй шаг цепочки; программа дрона уходит в сам дрон.

Сцена
левая часть окна: коврик с дронами и их путями, вид сверху (2D) или объёмный (3D).

Клетка
квадрат коврика 40×40 см; номер = столбец + 12 × строка.

Коврик
поле 12×12 клеток (4,8 м) с метками.

Лимит высоты
наибольшая разрешённая высота, от 20 до 200 см.

Миссия
программа одного дрона в виде набора шагов, который грузится в дрон.

Область
часть коврика, которую приложение считает рабочей: весь коврик или четверть.

Партитура
вкладка справа: временная лента, колонка на каждый борт, время сверху вниз, цвет светодиода линией, кружки на границах шагов.

Проверки
список замечаний симулятора к шоу.

Четверть
часть коврика 6×6 клеток; коврик сложен в четыре.

Шаг
одна команда в программе дрона (взлёт, переход, цвет…).

Шоу
набор дронов с их программами.

Симулятор
окно Litebee IDE, которое строит карту и партитуру по файлу шоу.

Старт
четвёртый шаг цепочки; дроны взлетают после подтверждения.

Статус «только симулятор»
шаг работает на экране, но на живом дроне не проверен.

Светодиод
цветная лампа на дроне; управляется командой `rgb`.

Ожидание
`set_z` на той же высоте: дрон просто висит.

Полуоборот
половина круга; `circle` считает круги полуоборотами.

Приложение D

Что ещё не проверено

Список для тех, кто готовит первый полёт.

- Шаг `code_z` в приложении упаковывается для загрузки так же, как в LiteBee Go, но на живом дроне его ещё никто не запускал. Шаг `circle` уже летал.
- Шаг `circle_z` в дрон загрузить нельзя.
- У `circle` на дроне шаг длится `count` × секунды, а симулятор считает секунды за весь шаг; направление `'cw'` на коврике в симуляторе угадано.
- Посадка в симуляции занимает 3 с, а в загруженной программе шаг посадки времени не добавляет.
- Свои цвета через `(r, g, b)` и `rgb_value` на живых дронах не проверялись: проверены только восемь именованных цветов.
- Модель симулятора приблизительная: дрон стартует под первой клеткой, летит по прямой с постоянной скоростью. Точность живого дрона 10–15 см.

Приложение Е

Ответы к упражнениям

Номер упражнения состоит из номера урока и номера внутри урока. У тех упражнений, где нужна программа, указан файл в папке `solutions`. Ваше решение может отличаться от нашего: главное, чтобы проверка проходила и шоу делало то, что требовалось.

Урок 1. Знакомство с окном

1.1. 30 секунд (в управлении временем «0,0 / 30,0 с»).

1.2. Жёлтый (`rgb('yellow')` в программе дрона 3; цвет виден и на карте, и в партитуре).

1.3. Клетка 49 (на вкладке «Код»: «Дрон 4 стоит на клетке 49»).

1.4. «Аварийная посадка: дронов на связи нет, остановлена симуляция.»

Урок 2. Первый взлёт

2.1. Нужно заменить `takeoff(100, 5)` на `takeoff(60, 4)`.

solutions/u02_1.py

```
# Упражнение 2.1: взлёт на 60 см за 4 секунды.
from litebee_show import Show

show = Show('Низкий взлёт')

d = show.drone(1)
d.calibrate(5)
d.takeoff(60, 4)
d.land()
```

2.2. Шоу длится $5 + 5 + 3 + 3 = 16$ секунд: `rgb` длится ноль секунд, а ожидание три.

solutions/u02_2.py

```
# Упражнение 2.2: зелёный свет и пауза 3 секунды перед посадкой.
from litebee_show import Show

show = Show('Зелёный огонёк')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('green')
d.set_z(100, 3)
d.rgb('black')
d.land()
```

2.3. $5 + 8 + 3 = 16$ секунд.

2.4. Нужно понизить высоту до 150 см или меньше, либо поднять лимит высоты до 180 см и выше (в окошке на верхней панели, потом снова «Проверить»).

solutions/u02_4.py

```
# Упражнение 2.4: взлёт ровно до лимита 150 см.
from litebee_show import Show

show = Show('Не выше лимита')

d = show.drone(1)
d.calibrate(5)
d.takeoff(150, 5)
d.land()
```

Урок 3. Свет

3.1. Шоу длится $5 + 5 + 3 + 1 + 3 + 3 = 20$ секунд (последние 3 — посадка).

solutions/u03_1.py

```
# Упражнение 3.1: красный 3 с, жёлтый 1 с, зелёный 3 с.
from litebee_show import Show

show = Show('Светофор')

d = show.drone(1)
d.calibrate(5)
d.takeoff(60, 5)
d.rgb('red')
d.set_z(60, 3)
d.rgb('yellow')
d.set_z(60, 1)
d.rgb('green')
d.set_z(60, 3)
d.rgb('black')
d.land()
```

3.2. Номера 1–7 — это красный, зелёный, жёлтый, синий, фиолетовый, голубой, белый (имена по порядку: `red`, `green`, `yellow`, `blue`, `purple`, `cyan`, `white`).

solutions/u03_2.py

```
# Упражнение 3.2: семь цветов по номерам, по секунде каждый.
from litebee_show import Show

show = Show('Радуга')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb(1)
d.set_z(100, 1)
d.rgb(2)
d.set_z(100, 1)
d.rgb(3)
d.set_z(100, 1)
d.rgb(4)
d.set_z(100, 1)
d.rgb(5)
d.set_z(100, 1)
d.rgb(6)
d.set_z(100, 1)
d.rgb(7)
d.set_z(100, 1)
d.rgb(0)
d.land()
```

3.3.

solutions/u03_3.py

```
# Упражнение 3.3: свой цвет через rgb_value.
from litebee_show import Show

show = Show('Оранжевый')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb_value(255, 128, 0)
d.set_z(100, 3)
d.rgb('black')
d.land()
```

3.4. Зритель увидит только зелёный. Красный включился и сразу сменился зелёным: между командами прошло ноль секунд.

solutions/u03_4.py

```
# Упражнение 3.4: красный не успевает показаться, rgb занимает 0 секунд.
from litebee_show import Show

show = Show('Два цвета подряд')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('red')
d.rgb('green')
d.set_z(100, 2)
d.rgb('black')
d.land()
```

Урок 4. Высота и ожидание

4.1.

solutions/u04_1.py

```
# Упражнение 4.1: ступени 50, 100, 150 см по две секунды.
from litebee_show import Show

show = Show('Лесенка вверх')

d = show.drone(1)
d.calibrate(5)
d.takeoff(50, 3)
d.set_z(100, 2)
d.set_z(150, 2)
d.land()
```

4.2.

solutions/u04_2.py

```
# Упражнение 4.2: между 60 и 120 см два раза.
from litebee_show import Show

show = Show('Дыхание')

d = show.drone(1)
d.calibrate(5)
d.takeoff(60, 5)
d.set_z(120, 2)
d.set_z(60, 2)
d.set_z(120, 2)
d.set_z(60, 2)
d.land()
```

4.3.

solutions/u04_3.py

```python
# Упражнение 4.3: десять секунд на 100 см, пять синих и пять красных.
from litebee_show import Show

show = Show('Десять секунд')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('blue')
d.set_z(100, 5)
d.rgb('red')
d.set_z(100, 5)
d.rgb('black')
d.land()
```

4.4.

solutions/u04_4.py

```python
# Упражнение 4.4: подняться до 150, спуститься до 30, снова до 100.
from litebee_show import Show

show = Show('Лифт')

d = show.drone(1)
d.calibrate(5)
d.takeoff(150, 5)
d.set_z(30, 4)
d.set_z(100, 3)
d.land()
```

Урок 5. Коврик и клетки

5.1. $3 + 12 \times 2 = 27$.

5.2. $40 = 12 \times 3 + 4$: столбец 4, строка 3.

5.3. Клетка 52: столбец 4, строка 4. Центр в точке $x = 160$ см, $y = 160$ см.

5.4. Между 13 и 16: $3 \times 40 = 120$ см. Между 13 и 52 по диагонали: $\sqrt{120^2 + 120^2} \approx 170$ см.

5.5. Клеток $6 \times 6 = 36$. Правая верхняя: столбец 5, строка 5, номер $5 + 60 = 65$.

5.6. Клетка 97 в строке 8, значит над ней строка 9: $1 + 12 \times 9 = 109$ (то есть 97 + 12).

Урок 6. Перелёт в клетку

6.1. Расстояние $2 \times 40 = 80$ см, скорость $80/3 \approx 27$ см/с.

solutions/u06_1.py

```python
# Упражнение 6.1: из 13 в 15 за 3 секунды.
from litebee_show import Show

show = Show('На две клетки')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(15, 3)
d.land()
```

6.2. Расстояние 120 см, за 1 секунду получается 120 см/с. Нужно время не меньше $120/100 = 1{,}2$ секунды; берём 4 секунды.

solutions/u06_2.py

```python
# Упражнение 6.2: тот же перелёт 13 -> 16, но не быстрее 100 см/с.
from litebee_show import Show

show = Show('Помедленнее')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 4)
d.land()
```

6.3. $5 + 5 + 1 + 4 + 4 + 3 = 22$ секунды.

solutions/u06_3.py

```python
# Упражнение 6.3: 13 -> 17 -> 13 по 4 секунды.
from litebee_show import Show

show = Show('Туда и обратно')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(17, 4)
d.code(13, 4)
d.land()
```

6.4. $170/100 = 1{,}7$ секунды. Округляем вверх до 2, но мы выбрали с запасом 4 секунды (скорость около 42 см/с).

solutions/u06_4.py

```
# Упражнение 6.4: 13 -> 52, 170 см, нужно больше 1,7 секунды.
from litebee_show import Show

show = Show('По диагонали')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(52, 4)
d.land()
```

Урок 7. Маршрут из нескольких клеток

7.1. Стороны 120, 120 и 170 см, времена 4, 4 и 6 секунд. Шоу длится $5 + 5 + 1 + 4 + 4 + 6 + 3 = 28$ секунд.

solutions/u07_1.py

```
# Упражнение 7.1: клетки 13, 16, 52 и обратно в 13.
from litebee_show import Show

show = Show('Треугольник')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 4)
d.code(52, 4)
d.code(13, 6)
d.land()
```

7.2. Стороны 120, 80, 120, 80 см; на четыре перехода 14 секунд, по 3,5. Шоу длится 28 секунд.

solutions/u07_2.py

```
# Упражнение 7.2: 3 клетки в ширину, 2 в высоту (клетки 13, 16, 40, 37).
from litebee_show import Show

show = Show('Прямоугольник')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code_s('16 40 37 13', 14)
d.land()
```

7.3.

solutions/u07_3.py

```python
# Упражнение 7.3: сверху вправо, по диагонали вниз, снизу вправо.
from litebee_show import Show

show = Show('Буква Z')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(49, 1)
d.code(52, 4)
d.code(13, 6)
d.code(16, 4)
d.land()
```

7.4.

solutions/u07_4.py

```python
# Упражнение 7.4: квадрат в обратную сторону, 13 -> 49 -> 52 -> 16 -> 13.
from litebee_show import Show

show = Show('Другая сторона')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code_s('49 52 16 13', 16)
d.land()
```

Урок 8. Посадка в клетку

8.1. Шоу длится $5 + 5 + 1 + 4 + 9 = 24$ секунды (`land_on` занимает 9 секунд).

solutions/u08_1.py

```python
# Упражнение 8.1: из 13 в 16 и посадка в центр клетки 16.
from litebee_show import Show

show = Show('Сесть у цели')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 4)
d.land_on(16)
```

8.2. Длительность увеличится на секунду: $3 + 1{,}5 + 2{,}5 + 3 = 10$ вместо 9, итого 25 секунд.

solutions/u08_2.py

```
# Упражнение 8.2: land_on с подлётом 3 секунды и остановкой на 60 см.
from litebee_show import Show

show = Show('Осторожная посадка')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 4)
d.land_on(16, 3, 60)
```

8.3. Результат тот же, 24 секунды.

solutions/u08_3.py

```
# Упражнение 8.3: то же, что land_on(16), но четырьмя командами.
from litebee_show import Show

show = Show('land_on по шагам')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(16, 4)
d.code(16, 2)
d.set_z(100, 1.5)
d.set_z(35, 2.5)
d.land()
```

8.4.

solutions/u08_4.py

```
# Упражнение 8.4: из 13 в 52 по диагонали и посадка в клетку 52.
from litebee_show import Show

show = Show('Диагональ и посадка')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(52, 4)
d.land_on(52)
```

Урок 9. Смотрим шоу: партитура

9.1. Команда `d.takeoff(100, 5)`: взлёт идёт с 5-й по 10-ю секунду (в программе дрона это третья строка).

9.2. Команда `d.code_s('16 52 49 13', 16)`: квадрат идёт с 11-й по 27-ю секунду.

9.3. На 19-й секунде: $11 + 4 + 4 = 19$.

9.4. 30 секунд.

9.5. Красный (в программе дрона 2 стоит `rgb('red')`).

Урок 10. Два дрона

10.1. Пути параллельны, горизонтальное расстояние между столбцами $3 \times 40 = 120$ см, что больше 60 см.

solutions/u10_1.py

```
# Упражнение 10.1: два дрона летят рядом вверх по столбцам 13 -> 49 и 16 -> 52.
from litebee_show import Show

show = Show('Параллельно')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(49, 6)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.code(16, 1)
d.code(52, 6)
d.land()
```

10.2.

solutions/u10_2.py

```
# Упражнение 10.2: диагонали 13 -> 52 и 16 -> 49, высоты 80 и 150 см.
from litebee_show import Show

show = Show('Крест')

d = show.drone(1)
d.calibrate(5)
d.takeoff(80, 5)
d.code(13, 1)
d.code(52, 7)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(150, 5)
d.code(16, 1)
d.code(49, 7)
d.land()
```

10.3. Квадрат на 80 см, висящий дрон на 150 см: зазор 70 см.

solutions/u10_3.py

```
# Упражнение 10.3: один летит по квадрату, второй висит выше и мигает.
from litebee_show import Show

show = Show('Облёт вокруг лампы')

d = show.drone(1)
d.calibrate(5)
d.takeoff(80, 5)
d.rgb('green')
d.code(13, 1)
d.code_s('16 52 49 13', 16)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(150, 5)
d.code(27, 1)
d.rgb('red')
d.set_z(150, 2)
d.rgb('black')
d.set_z(150, 2)
d.rgb('red')
d.set_z(150, 2)
d.rgb('black')
d.set_z(150, 2)
d.rgb('red')
d.set_z(150, 2)
d.rgb('black')
d.set_z(150, 2)
d.rgb('red')
d.set_z(150, 2)
d.rgb('black')
d.set_z(150, 2)
d.land()
```

10.4. Дроны летят на разных высотах: 80 и 150 см.

solutions/u10_4.py

```
# Упражнение 10.4: исправленный файл bad_close.py, высоты 80 и 150 см.
from litebee_show import Show

show = Show('Без сближения')

d = show.drone(1)
d.calibrate(5)
d.takeoff(80, 5)
d.code(13, 1)
d.code(16, 4)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(150, 5)
d.code(15, 1)
d.code(12, 4)
d.land()
```

Урок 11. Проверки

11.1. Понижены высоты до 80 и 150 см, дрону 1 добавлено время (4 секунды вместо 1), дроны разведены по высоте.

solutions/u11_1.py

```
# Упражнение 11.1: исправленный bad_three.py.
from litebee_show import Show

show = Show('Три проблемы решены')

d = show.drone(1)
d.calibrate(5)
d.takeoff(80, 5)
d.code(13, 1)
d.code(16, 4)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(150, 5)
d.code(15, 1)
d.code(13, 6)
d.land()
```

11.2. Поднять лимит высоты до 180 см или больше (до 200 см) и снова нажать «Проверить». Файл менять не нужно.

11.3. Разбить ожидание на два шага по 45 секунд: каждый короче 60.

solutions/u11_3.py

```
# Упражнение 11.3: 90 секунд ожидания разбиты на два шага по 45.
from litebee_show import Show

show = Show('Ожидание по частям')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.set_z(100, 45)
d.set_z(100, 45)
d.land()
```

11.4. Клетка 8 лежит в столбце 8, а четверть 1 — это столбцы 0–5. Заменяем на клетку 5.

solutions/u11_4.py

```
# Упражнение 11.4: клетка 8 заменена на клетку 5.
from litebee_show import Show

show = Show('Внутри четверти')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(5, 4)
d.land()
```

Урок 12. Очередь и ожидание

12.1. Дрон 2 ждёт 9 секунд (8 секунд полёта дрона 1 и запас).

solutions/u12_1.py

```
# Упражнение 12.1: сначала летит дрон 1, потом дрон 2.
from litebee_show import Show

show = Show('Два по очереди')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(49, 4)
d.code(13, 4)
d.set_z(100, 9)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.code(15, 1)
d.set_z(100, 9)
d.code(51, 4)
d.code(15, 4)
d.land()
```

12.2.

solutions/u12_2.py

```
# Упражнение 12.2: три дрона зажигаются по очереди на две секунды.
from litebee_show import Show

show = Show('Огни по очереди')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.rgb('red')
d.set_z(100, 2)
d.rgb('black')
d.set_z(100, 4)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.code(15, 1)
d.set_z(100, 2)
d.rgb('red')
d.set_z(100, 2)
d.rgb('black')
d.set_z(100, 2)
d.land()

d = show.drone(3)
d.calibrate(5)
d.takeoff(100, 5)
d.code(17, 1)
d.set_z(100, 4)
d.rgb('red')
d.set_z(100, 2)
d.rgb('black')
d.land()
```

12.3.

solutions/u12_3.py

```
# Упражнение 12.3: три дрона по очереди поднимаются до 150 см и опускаются до 80.
from litebee_show import Show

show = Show('Волна')

d = show.drone(1)
d.calibrate(5)
d.takeoff(80, 5)
d.code(13, 1)
d.set_z(150, 2)
d.set_z(80, 2)
d.set_z(80, 4)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(80, 5)
d.code(15, 1)
d.set_z(80, 2)
d.set_z(150, 2)
d.set_z(80, 2)
d.set_z(80, 2)
d.land()

d = show.drone(3)
d.calibrate(5)
d.takeoff(80, 5)
d.code(17, 1)
d.set_z(80, 4)
d.set_z(150, 2)
d.set_z(80, 2)
d.land()
```

12.4. «Позже» записывается ожиданием: перед движением `set_z(100, 3)` на той же высоте.

solutions/u12_4.py

```
# Упражнение 12.4: дрон 2 летит по соседней линии на 3 секунды позже дрона 1.
from litebee_show import Show

show = Show('Опоздавший')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
d.code(17, 6)
d.set_z(100, 3)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.code(37, 1)
d.set_z(100, 3)
d.code(41, 6)
d.land()
```

Урок 13. Повторы: циклы

13.1.

solutions/u13_1.py

```python
# Упражнение 13.1: три раза туда и обратно между 13 и 17.
from litebee_show import Show

show = Show('Челнок')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
for _ in range(3):
    d.code(17, 4)
    d.code(13, 4)
d.land()
```

13.2.

solutions/u13_2.py

```python
# Упражнение 13.2: семь цветов списком и циклом.
from litebee_show import Show

show = Show('Радуга циклом')

d = show.drone(1)
colors = ['red', 'yellow', 'green', 'cyan', 'blue', 'purple', 'white']
d.calibrate(5)
d.takeoff(100, 5)
for color in colors:
    d.rgb(color)
    d.set_z(100, 1)
d.rgb('black')
d.land()
```

13.3. Дрон `n` ждёт `2 * n` секунд до своей вспышки и `2 * (2 - n)` после неё, поэтому длина у всех одинаковая.

solutions/u13_3.py

```
# Упражнение 13.3: три дрона мигают по очереди, пауза зависит от номера.
from litebee_show import Show

show = Show('Очередь циклом')

cells = [13, 15, 17]

for n in range(3):
    d = show.drone(n + 1)
    d.calibrate(5)
    d.takeoff(100, 5)
    d.code(cells[n], 1)
    d.set_z(100, 2 * n)
    d.rgb('red')
    d.set_z(100, 2)
    d.rgb('black')
    d.set_z(100, 2 * (2 - n))
    d.land()
```

13.4.

solutions/u13_4.py

```
# Упражнение 13.4: квадрат три раза подряд.
from litebee_show import Show

show = Show('Три круга')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(13, 1)
for lap in range(3):
    d.code_s('16 52 49 13', 16)
d.land()
```

Урок 14. Другая четверть

14.1. Клетки 13, 16, 52, 49 плюс 72: 85, 88, 124, 121.

solutions/u14_1.py

```
# Упражнение 14.1: квадрат из урока 7, к каждой клетке добавлено 72.
from litebee_show import Show

show = Show('Третья четверть')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(85, 1)
d.code_s('88 124 121 85', 16)
d.land()
```

14.2. Клетки 13, 16, 52, 49 плюс 78: 91, 94, 130, 127.

solutions/u14_2.py

```
# Упражнение 14.2: квадрат из урока 7, к каждой клетке добавлено 78.
from litebee_show import Show

show = Show('Четвёртая четверть')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(91, 1)
d.code_s('94 130 127 91', 16)
d.land()
```

14.3. $14 + 72 = 86$; $55 + 78 = 133$.

14.4. $100 = 12 \times 8 + 4$: столбец 4, строка 8. Столбцы 0–5 и строки 6–11: это четверть 3. В четверти 1 то же место имеет номер $100 - 72 = 28$.

Урок 15. Круги и подъём на ходу

15.1. Меняем `'cw'` на `'ccw'`.

solutions/u15_1.py

```
# Упражнение 15.1: полный круг в другую сторону.
from litebee_show import Show

show = Show('Против часовой')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(14, 1)
d.circle(2, 26, 26, 12, 'ccw')
d.land()
```

15.2. Дрон оказывается в клетке 38 (на противоположной стороне круга: центр клетки 26, клетка 14 снизу, клетка 38 сверху).

solutions/u15_2.py

```
# Упражнение 15.2: половина круга, дрон оказывается напротив, в клетке 38.
from litebee_show import Show

show = Show('Полкруга')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(14, 1)
d.circle(1, 26, 26, 6)
d.land_on(38)
d.land()
```

15.3.

solutions/u15_3.py

```
# Упражнение 15.3: подъём с 60 до 140 см по пути из 13 в 16 и спуск обратно.
from litebee_show import Show

show = Show('Горка')

d = show.drone(1)
d.calibrate(5)
d.takeoff(60, 5)
d.code(13, 1)
d.code_z(16, 140, 6)
d.code_z(13, 60, 6)
d.land()
```

15.4. Дроны стоят на противоположных концах диаметра длиной 80 см и крутятся с одной скоростью в одну сторону, расстояние между ними всё время 80 см, больше порога 60 см.

solutions/u15_4.py

```
# Упражнение 15.4: два дрона по кругу вокруг клетки 26, один напротив другого.
from litebee_show import Show

show = Show('Танец вдвоём')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.code(14, 1)
d.circle(2, 26, 26, 12)
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.code(38, 1)
d.circle(2, 26, 26, 12)
d.land()
```

Урок 16. Итоговые проекты

Проект 1.

solutions/t06_follow.py

```
# Проект 1. Дрон 2 летит по следам лидера на один переход (4 секунды) позже и выше: 150 против 80 см.
from litebee_show import Show

show = Show('За лидером')

d = show.drone(1)
d.calibrate(5)
d.takeoff(80, 5)
d.rgb('red')
d.code(13, 1)
d.code(16, 4)
d.code(52, 4)
d.code(49, 4)
d.code(13, 4)
d.rgb('black')
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(150, 5)
d.rgb('blue')
d.code(37, 1)
d.code(13, 4)
d.code(16, 4)
d.code(52, 4)
d.code(49, 4)
d.code(13, 4)
d.code(37, 4)
d.rgb('black')
d.land_on(37)
```

Проект 2.

solutions/t09_ladder.py

```
# Проект 2. Лесенка: три дрона на высотах 20, 85 и 150 см (зазор не меньше 60 см) сдвигаются по кругу.
from litebee_show import Show

show = Show('Лесенка')

d = show.drone(1)
d.calibrate(5)
d.takeoff(20, 5)
d.rgb('red')
d.code(13, 1)
d.code(15, 5)
d.rgb('black')
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(85, 5)
d.rgb('yellow')
d.code(15, 1)
d.code(17, 5)
d.rgb('black')
d.land()

d = show.drone(3)
d.calibrate(5)
d.takeoff(150, 5)
d.rgb('green')
d.code(17, 1)
d.code(13, 5)
d.rgb('black')
d.land()
```

Проект 3.

solutions/t07_corners.py

```
# Проект 3. Четыре дрона стоят по углам квадрата 5 х 5 клеток и обходят его по кругу.
from litebee_show import Show

show = Show('Углы по кругу')

d = show.drone(1)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('red')
d.code(0, 1)
d.code(4, 4)
d.code(52, 4)
d.code(48, 4)
d.code(0, 4)
d.rgb('black')
d.land()

d = show.drone(2)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('yellow')
d.code(4, 1)
d.code(52, 4)
d.code(48, 4)
d.code(0, 4)
d.code(4, 4)
d.rgb('black')
d.land()

d = show.drone(3)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('green')
d.code(52, 1)
d.code(48, 4)
d.code(0, 4)
d.code(4, 4)
d.code(52, 4)
d.rgb('black')
d.land()

d = show.drone(4)
d.calibrate(5)
d.takeoff(100, 5)
d.rgb('blue')
d.code(48, 1)
d.code(0, 4)
d.code(4, 4)
d.code(52, 4)
d.code(48, 4)
d.rgb('black')
d.land()
```

Проект 4.

solutions/t15_star5.py

```python
# Проект 4. Пять дронов: четыре по углам обходят квадрат, пятый в центре поднимается и опускается.
from litebee_show import Show

show = Show('Пятёрка')

corners = [0, 4, 52, 48]
colors = ['red', 'yellow', 'green', 'blue']

for n in range(4):
    d = show.drone(n + 1)
    d.calibrate(5)
    d.takeoff(100, 5)
    d.rgb(colors[n])
    d.code(corners[n], 1)
    d.code(corners[(n + 1) % 4], 5)
    d.code(corners[(n + 2) % 4], 5)
    d.rgb('black')
    d.land()

d = show.drone(5)
d.calibrate(5)
d.takeoff(60, 5)
d.rgb('white')
d.code(26, 1)
d.set_z(120, 5)
d.set_z(60, 5)
d.rgb('black')
d.land()
```

Проект 5.

solutions/t16_rows.py

```
# Проект 5. Шесть дронов в двух рядах меняются рядами: нижний ряд летит на 80 см, верхний на 150 см.
from litebee_show import Show

show = Show('Ряды меняются')

bottom = [13, 15, 17]
top = [49, 51, 53]

for i in range(3):
    d = show.drone(i + 1)
    d.calibrate(5)
    d.takeoff(80, 5)
    d.rgb('blue')
    d.code(bottom[i], 1)
    d.code(top[i], 6)
    d.rgb('black')
    d.land_on(top[i])

    d = show.drone(i + 4)
    d.calibrate(5)
    d.takeoff(150, 5)
    d.rgb('red')
    d.code(top[i], 1)
    d.code(bottom[i], 6)
    d.rgb('black')
    d.land_on(bottom[i])
```

Проект 6.

solutions/t17_hex7.py

```python
# Проект 6. Семь дронов: шестёрка по кольцу сдвигается на два места, седьмой в центре мигает.
from litebee_show import Show

show = Show('Семёрка')

ring = [2, 16, 40, 50, 36, 12]
colors = ['red', 'yellow', 'green', 'cyan', 'blue', 'purple']

for n in range(6):
    d = show.drone(n + 1)
    d.calibrate(5)
    d.takeoff(90 if n % 2 == 0 else 130, 5)
    d.rgb(colors[n])
    d.code(ring[n], 1)
    d.code(ring[(n + 1) % 6], 5)
    d.code(ring[(n + 2) % 6], 5)
    d.rgb('black')
    d.land()

d = show.drone(7)
d.calibrate(5)
d.takeoff(60, 5)
d.code(26, 1)
for _ in range(5):
    d.rgb('white')
    d.set_z(60, 1)
    d.rgb('black')
    d.set_z(60, 1)
d.land()
```

Проект 7.

solutions/t18_eight.py

```python
# Проект 7. Восемь дронов по кольцу: чётные дроны идут по кольцу в одну сторону, нечётные навстречу, на два места.
from litebee_show import Show

show = Show('Встречный хоровод')

ring = [0, 2, 4, 28, 52, 50, 48, 24]

for i in range(8):
    high = i % 2 == 0
    step = 1 if high else -1
    way = [ring[(i + step * k) % 8] for k in range(3)]
    d = show.drone(i + 1)
    d.calibrate(5)
    d.takeoff(150 if high else 80, 5)
    d.rgb('yellow' if high else 'cyan')
    d.code(way[0], 1)
    d.code(way[1], 5)
    d.code(way[2], 5)
    d.rgb('black')
    d.land_on(way[2])
```